Détection et segmentation

Jour 2 · Application au deep learning

ING3 option IA · CY Tech



Du Jour 1 au Jour 2 : classer, détecter, segmenter

Pourquoi : le Jour 1 répondait à « qu'y a-t-il dans l'image ? » ; aujourd'hui, « où ? » et « quels pixels ? ».



Classification

Une étiquette pour l'image entière. Jour 1.



Détection

Pour chaque objet : une boîte, une classe, un score. Combien et où.



Segmentation

Une classe pour chaque pixel, ou un masque par objet. La forme exacte.

Image « astronaut » de scikit-image ; détection YOLOv8n (COCO, 416 px), masque MobileSAM à partir de la boîte, calculés pour le cours.

Le cas d'étude de la séance

Pourquoi : toutes les notions du jour s'appliquent à ce cas, le matin comme au TD.

Repérer les personnes sans casque sur un chantier

- **Données** : Construction-PPE, images de chantier annotées en boîtes, 12 classes (casque, gilet, gants, personne, absence de casque…)
- **Matériel** : les machines de l'école, sans GPU
- **À rendre** : un détecteur et un seuil d'alerte, justifiés par des mesures

Les trois questions traitées par la séance

1. **Comment localiser les objets ?** — parties 1 et 2
   des boîtes, l'IoU, la NMS, et ce que calcule YOLO
2. **Comment savoir si le détecteur est bon ?** — parties 3 et 4
   mesurer, choisir un seuil, fine-tuner, comparer à une baseline
3. **Faut-il des boîtes ou des masques ?** — parties 5 et 6
   Faster R-CNN, segmentation sémantique et d'instances, SAM

Puis la même démarche sur les tôles d'acier du projet (brique 2) : localiser le défaut, et non plus seulement le reconnaître.

Construction-PPE : Ultralytics, docs.ultralytics.com/datasets/detect/construction-ppe.

Objectifs de la séance

Pourquoi : ce que vous devez savoir faire à la fin de la journée.

À la fin de la journée, vous saurez **localiser des objets dans une image, mesurer la qualité d'un détecteur et choisir entre boîtes et masques**. En détail :

Expliquer

comment YOLO passe d'une image à des boîtes : grilles, distances aux bords, scores, NMS

Mesurer

un détecteur : précision, rappel, AP, mAP, et choisir un seuil selon le coût des erreurs

Adapter

un détecteur à de nouvelles classes par fine-tuning, et en estimer le coût

Choisir

entre détection, segmentation sémantique et segmentation d'instances, avec les modèles adaptés

Prérequis de la séance

Pourquoi : ce que le cours suppose connu, et où le revoir.

Ce que le cours suppose connu	Où le revoir
CNN : convolution, cartes de features, backbone et tête	Jour 1, partie 1
Fine-tuning : remplacer la tête, geler ou réentraîner les étages	Jour 1, parties 6 et 7
Validation et test ; précision et rappel en classification	Jour 1, partie 5 et diagnostic
YOLO, dans les grandes lignes	introduction au deep learning ; repris en partie 2

Non supposés connus : la segmentation, Faster R-CNN et SAM, présentés aujourd'hui.

Ce que l'on va voir ce matin, et pourquoi

Pourquoi : le cours suit l'ordre du TD : chaque partie prépare une ou deux étapes de l'après-midi.

9h30 · avant la pause

Partie 1 — 1

Boîtes, IoU, NMS

les outils de base

TD · étapes 2 à 4

Partie 2 — 1

Comment YOLO prédit

grilles, sorties, Ultralytics

TD · étape 4

Partie 3 — 2

Mesurer un détecteur

AP, mAP, baseline, seuil

TD · étapes 5 à 7

11h · après la pause

Partie 4 — 2

Fine-tuner YOLO

nos 12 classes

TD · étape 6

Partie 5 — 3

Faster R-CNN

l'approche en deux étapes

TD · pour aller plus loin

Partie 6 — 3

Segmentation

U-Net, DeepLab, SAM

TD · étape 8

Questions du cas d'étude : 1 comment localiser les objets ; 2 comment savoir si le détecteur est bon ; 3 faut-il des boîtes ou des masques. Étapes 9 à 11 du TD (mini-application, fiche de décision, brique 2 du projet) : voir le déroulé en fin de cours. Pause vers 10h50.



Partie 1

Boîtes, IoU et NMS

Pourquoi : écrire une boîte, puis les deux outils (IoU et NMS) dont dépend tout détecteur, codés par vous à l'étape 2 du TD

La détection d'objets : définition

Définition

Un détecteur d'objets renvoie, pour chaque objet de l'image, une boîte qui l'encadre, sa classe et un score de confiance entre 0 et 1. Leur nombre varie d'une image à l'autre.

Le problème que cela résout

Un classifieur dit ce que contient l'image, pas où ni combien. Sur un chantier, il faut savoir où est chaque personne et si chacune porte un casque.

Ce qui entre, ce qui sort

Entrée : une image. Sortie : une liste de détections (boîte [x1, y1, x2, y2], classe, score), de longueur variable.

Ce que l'on va voir, dans l'ordre

2. Un détecteur, de l'image aux boîtes : l'exemple de YOLO
3. Représenter une boîte
4. Boîtes en PyTorch
5. L'IoU (Intersection over Union)
6. IoU en PyTorch
7. La NMS (Non-Maximum Suppression) : une boîte par objet
8. La NMS pas à pas
9. NMS en PyTorch
10. À retenir

Dans le TD : Étape 2 : vous codez l'IoU (TODO 1) et la NMS (TODO 2), les deux outils de base.

Pour en savoir plus : d2l, section 14.3 : Object Detection and Bounding Boxes ; d2l, section 14.4 : Anchor Boxes (IoU, NMS)

Un détecteur, de l'image aux boîtes : l'exemple de YOLO

Pourquoi : voir le chemin complet d'un détecteur avant d'en étudier chaque pièce.

Un **détecteur** reçoit une image et rend une liste d'objets : pour chacun, une **boîte**, une **classe** et un **score** de confiance entre 0 et 1. Voici comment YOLO, le détecteur du TD, y arrive, en quatre temps.

1. l'image



l'image du chantier

en entrée

2. score ≥ 0,05 : 215 boîtes



une boîte en chacun des 3 549 points ; ici, les 215 de score ≥ 0,05

partie 2

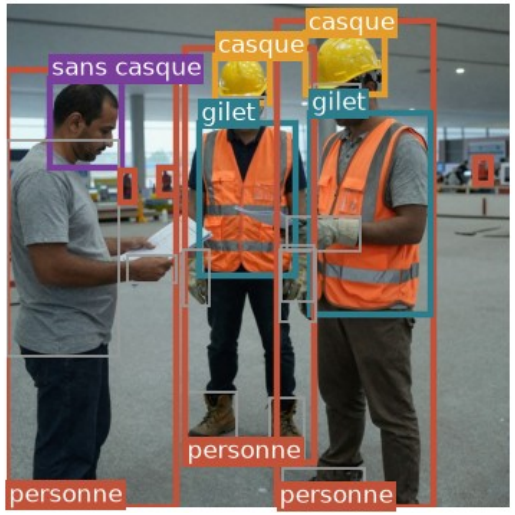
3. score ≥ 0,25 : 151



on garde celles dont le score dépasse un seuil

partie 3

4. après la NMS : 22



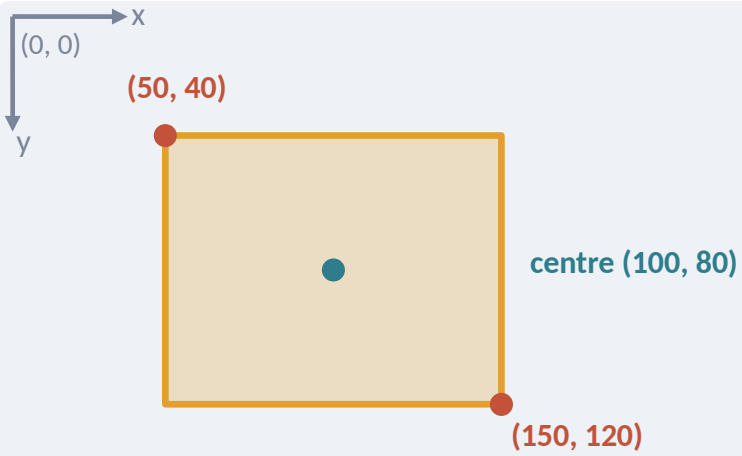
la NMS supprime les doublons : une boîte par objet

cette partie

Source : Modèle fine-tuné du TD (YOLOv8n, Construction-PPE), image de validation, NMS par classe au seuil 0,7. Calculé pour le cours.

Représenter une boîte

Pourquoi : trois écritures de la même boîte circulent entre les bibliothèques ; une confusion décale toutes les boîtes.



quart haut gauche d'une image 640 × 480

Coins (xyxy)

```
[50, 40, 150, 120]
```

torchvision, la NMS, l'IoU

Centre et taille (cxcywh)

```
[100, 80, 100, 80]
```

la sortie de nombreux détecteurs

Format YOLO (fichier .txt)

```
0  0,15625  0,16667  0,15625  0,16667
```

classe, puis centre et taille divisés par 640 et 480

Pixels, origine en haut à gauche, y vers le bas. Aire d'une boîte : (x2 − x1) × (y2 − y1).

Source : Ultralytics, format des jeux de détection ; documentation torchvision, box_convert.

Boîtes en PyTorch

Pourquoi : convertir sans se tromper, et lire les fichiers d'annotation du TD.

```python
import torch
from torchvision.ops import box_convert

boite = torch.tensor([[50., 40., 150., 120.]])     # coins [x1, y1, x2, y2]
print(box_convert(boite, "xyxy", "cxcywh"))        # centre et taille
# tensor([[100.,  80., 100.,  80.]])

# une ligne de fichier YOLO : classe cx cy w h, divisés par 640 et 480
ligne = "0 0.15625 0.16667 0.15625 0.16667"
classe, *v = ligne.split()
cx, cy, w, h = [float(a) * t for a, t in zip(v, (640, 480, 640, 480))]
pixels = torch.tensor([[cx, cy, w, h]])
print(classe, box_convert(pixels, "cxcywh", "xyxy").round())
# 0 tensor([[ 50.,  40., 150., 120.]])
```

- **box_convert** passe d'une écriture à l'autre, pour un tenseur (N, 4) de boîtes
- Une ligne YOLO : on **multiplie** le centre et la taille par la largeur et la hauteur de l'image
- Au TD, **tp.lire_annotations** fait cette conversion pour vous

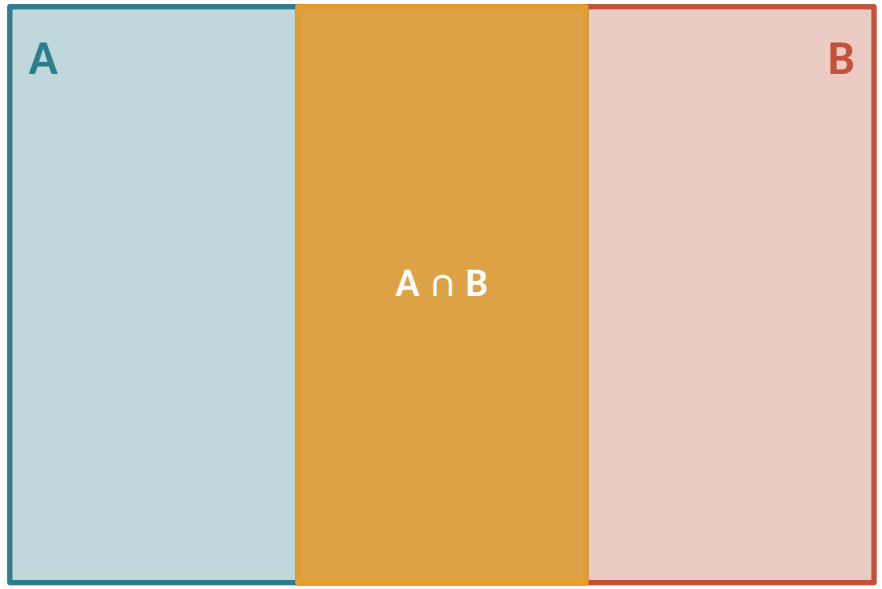
Tous les extraits de code du cours ont été exécutés avec les versions de uv.lock (PyTorch 2.14.1) : les sorties écrites en commentaire sont celles obtenues.

Source : Documentation torchvision : torchvision.ops.box_convert.

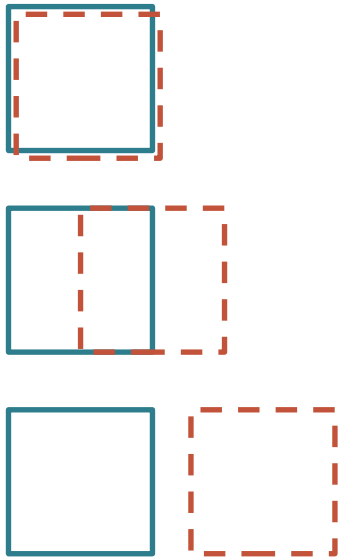
L'IoU (Intersection over Union)

Pourquoi : c'est la mesure qui dit si une boîte prédite correspond à un objet : toute l'évaluation en dépend.

IoU veut dire **Intersection over Union** (« intersection sur union »). C'est un nombre entre **0** et **1** qui dit à quel point deux boîtes se recouvrent : il sert à décider si une boîte prédite tombe bien sur un objet.



- Aire de A : 10 × 10 = **100** ; aire de B : **100**
- Intersection (orange) : 5 × 10 = **50**
- Union : 100 + 100 − 50 = **150**
- IoU = 50 / 150 = **0,33**



identiques : IoU 1

décalées de moitié : IoU 0,33

disjointes : IoU 0

Une détection compte comme **juste** si sa classe est la bonne et si son **IoU avec un objet annoté dépasse 0,5** (Pascal VOC ; COCO et Ultralytics : IoU ≥ 0,5). **Pascal VOC** (20 classes) et **COCO** (80 classes d'objets courants) : les deux jeux de données de référence de la détection.

Source : Everingham et al., The PASCAL Visual Object Classes Challenge, IJCV 2010 (critère IoU > 0,5).

IoU en PyTorch

Pourquoi : calculer d'un coup l'IoU entre toutes les prédictions et toutes les vérités.

```python
import torch
from torchvision.ops import box_iou

predites = torch.tensor([[0., 0., 10., 10.]])
verites = torch.tensor([[5., 0., 15., 10.],      # décalée de moitié
                        [0., 0., 10., 10.],      # identique
                        [20., 20., 30., 30.]])   # disjointe
print(box_iou(predites, verites))   # (prédictions, vérités)
# tensor([[0.3333, 1.0000, 0.0000]])
```

- **box_iou(a, b)** : toutes les paires d'un coup, une matrice (N, M)
- Une ligne par prédiction, une colonne par vérité : c'est le tableau dont part l'appariement
- Au TD (TODO 1), vous codez l'IoU de deux boîtes, **sans appeler box_iou** : les tests la comparent à des valeurs calculées à la main

Source : Documentation torchvision : torchvision.ops.box_iou.

La NMS (Non-Maximum Suppression) : une boîte par objet

Pourquoi : voir l'effet de la NMS sur la sortie réelle d'un détecteur.

NMS veut dire **Non-Maximum Suppression** (« suppression des non-maxima »). Un détecteur propose souvent **plusieurs boîtes pour le même objet** ; la NMS n'en garde qu'**une** : la mieux notée, et supprime celles qui la recouvrent trop (IoU au-dessus d'un seuil).

avant la NMS : 17 boîtes



après la NMS (seuil 0,5) : 2



- Avant : **17** boîtes « personne » (score ≥ 0,05), presque toutes sur la même personne
- Après la NMS au seuil d'IoU 0,5 : **2** boîtes
- La seconde (score **0,07**) encadre le casque spatial : la NMS supprime les **doublons**, pas les fausses alertes ; c'est le **seuil de confiance** qui l'écarte

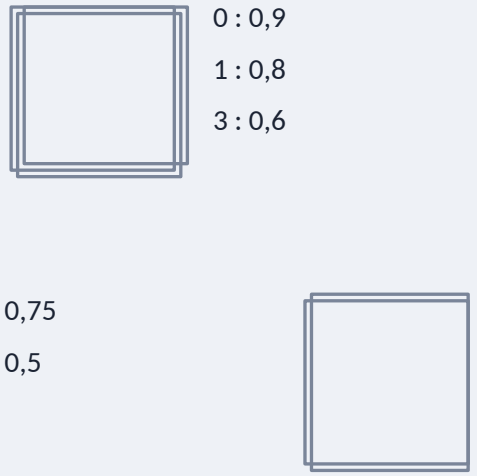
Source : YOLOv8n (COCO) à 416 px, NMS de torchvision ; image « astronaut » de scikit-image. Calculé pour le cours.

La NMS pas à pas

Pourquoi : sans elle, chaque objet serait détecté plusieurs fois.

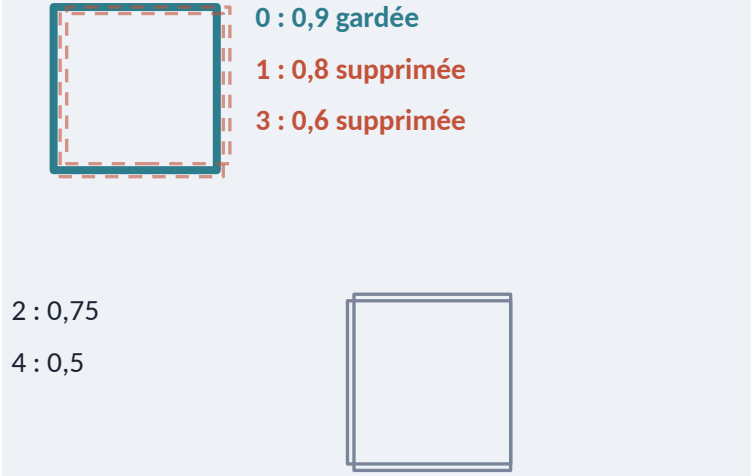
L'algorithme, sur 5 boîtes et 2 objets, seuil d'IoU **0,5** : on garde la boîte de **meilleur score**, on supprime celles qui la recouvrent à plus de 0,5, puis on **recommence** avec celles qui restent.

Départ : 5 boîtes, triées par score



ordre : 0 (0,9), 1 (0,8), 2 (0,75), 3 (0,6), 4 (0,5)

Tour 1 : garder 0 ; supprimer 1 et 3



IoU avec 0 : 1 → 0,85 ; 3 → 0,82 ; plus de 0,5

Tour 2 : garder 2 ; supprimer 4



IoU avec 2 : 4 → 0,89. Résultat : [0, 2]

Une boîte est supprimée si son IoU avec une boîte gardée est **strictement supérieure** au seuil ; la boîte gardée sort de la liste à chaque tour.

Source : Exemple calculé avec torchvision.ops.nms (diapositive suivante).

NMS en PyTorch

Pourquoi : la NMS toute faite, et sa version classe par classe, celle qu'utilise YOLO.

```python
import torch
from torchvision.ops import nms, batched_nms

boites = torch.tensor([[10., 10., 60, 60], [12, 12, 62, 62],
                       [100, 100, 150, 150], [14, 10, 64, 58],
                       [102, 98, 150, 152]])
scores = torch.tensor([0.9, 0.8, 0.75, 0.6, 0.5])
print(nms(boites, scores, iou_threshold=0.5))    # tensor([0, 2])

# NMS classe par classe : la boîte 1 est d'une autre classe
classes = torch.tensor([0, 1, 0, 0, 0])
print(batched_nms(boites, scores, classes, iou_threshold=0.5))
# tensor([0, 1, 2]) : la boîte 1 est gardée
```

- **nms(boites, scores, seuil)** : les indices gardés, par score décroissant
- **batched_nms** ajoute les classes : deux boîtes de classes différentes ne se suppriment pas
- Ultralytics applique la NMS **classe par classe**, avec un seuil de 0,7 par défaut
- Au TD (TODO 2), recodez-la **sans appeler nms** : torchvision sert de référence au test

Source : Documentation torchvision : torchvision.ops.nms et batched_nms.

Question à la salle

Deux casques côte à côte, boîtes d'IoU 0,4 : que fait une NMS au seuil 0,3 ? à 0,7 ?



La détection d'objets : à retenir

1. Une détection, c'est une **boîte**, une **classe** et un **score** ; attention aux trois écritures d'une boîte (coins, centre et taille, format YOLO normalisé).

2. L'**IoU** mesure le recouvrement de deux boîtes : aire de l'intersection divisée par aire de l'union. Critère classique : une détection est correcte si sa classe est la bonne et si son IoU dépasse 0,5 (≥ 0,5 chez COCO et Ultralytics).

3. La **NMS** garde la meilleure boîte et supprime celles qui la recouvrent trop, puis recommence ; YOLO l'applique **classe par classe**.

Dans le TD : Étape 2 : vous codez l'IoU (TODO 1) et la NMS (TODO 2), les deux outils de base.

Pour approfondir : d2l, section 14.3 : Object Detection and Bounding Boxes ; d2l, section 14.4 : Anchor Boxes (IoU, NMS)



Partie 2

Comment YOLO prédit les boîtes

Pourquoi : savoir ce que calcule le détecteur du TD, pour savoir ce que l'on règle et ce que l'on fine-tune

YOLO : définition

Définition

YOLO (You Only Look Once) est une famille de détecteurs en une seule passe : un CNN calcule des cartes de features, puis une tête prédit, en chaque point de grilles (trois depuis YOLOv3), une boîte et un score par classe. Une NMS fait ensuite le tri.

Le problème que cela résout

Détecter en temps réel, ou presque, y compris sur des machines modestes : une seule passe du réseau par image, sans étape de propositions de régions.

Ce qui entre, ce qui sort

Entrée : une image redimensionnée (416 × 416 au TD). Sortie brute : 4 valeurs de boîte + 80 scores (un par classe COCO) pour chacun des 3 549 points des trois grilles ; sortie finale : les détections après seuil et NMS.

Pour en savoir plus : Documentation Ultralytics : YOLOv8 ; Redmon et al., 2016 : l'article YOLO (arXiv)

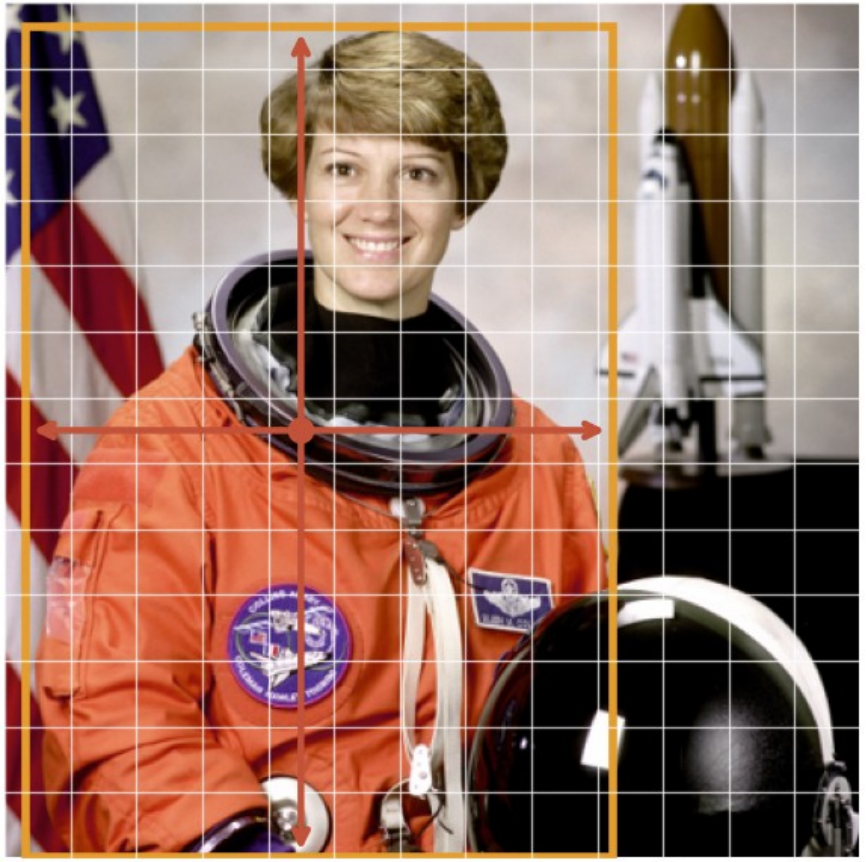
Ce que l'on va voir, dans l'ordre

2. Le principe one-stage
3. Trois grilles, trois échelles
4. Ce que prédit chaque point
5. YOLO en PyTorch : ce que sort le réseau
6. YOLO en PyTorch : predict, l'appel du TD
7. Cinq tailles de YOLOv8 : n, s, m, l, x
8. À retenir

Dans le TD : Étape 4 : YOLOv8n pré-entraîné sur COCO face aux images de chantier ; étape 6 : son fine-tuning.

Le principe one-stage

Pourquoi : comprendre ce que calcule YOLO, pour savoir ce que l'on règle et ce que l'on fine-tune.



grille de pas 32 à 416 px (13 × 13) ; en rouge, un point et ses distances aux bords de la boîte

Backbone — un CNN, comme au Jour 1 : l'image devient des cartes de features

↓

Cou (neck) — mélange les cartes de trois résolutions (FPN, Feature Pyramid Network) : petits et grands objets

↓

Tête — en chaque point des trois grilles : une boîte et un score par classe

↓

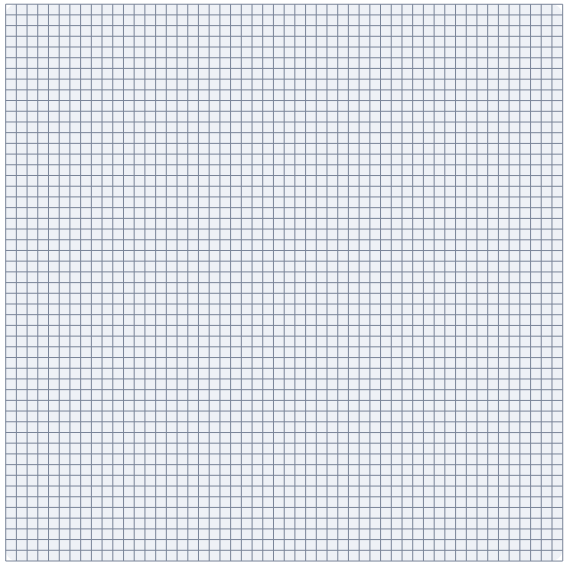
Post-traitement — seuil de confiance, puis NMS par classe : les détections finales

Une seule passe du réseau : pas d'étape séparée qui propose des régions (c'est la différence avec Faster R-CNN, partie 5).

Source : Redmon et al., You Only Look Once, CVPR 2016 ; documentation Ultralytics YOLOv8.

Trois grilles, trois échelles

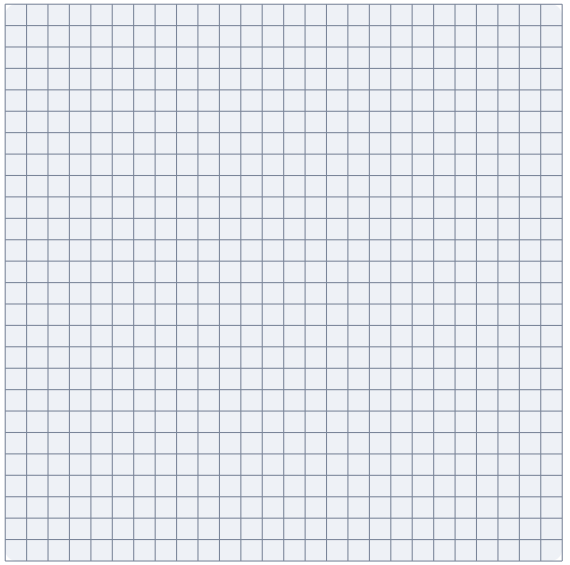
Pourquoi : c'est ce qui permet de détecter à la fois de petits et de grands objets.



pas 8 : un point tous les 8 px

52 × 52 = 2 704 points

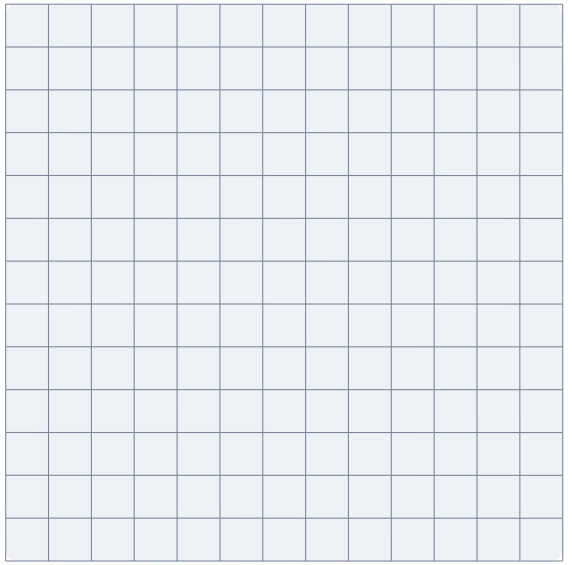
petits objets



pas 16 : tous les 16 px

26 × 26 = 676 points

objets moyens



pas 32 : tous les 32 px

13 × 13 = 169 points

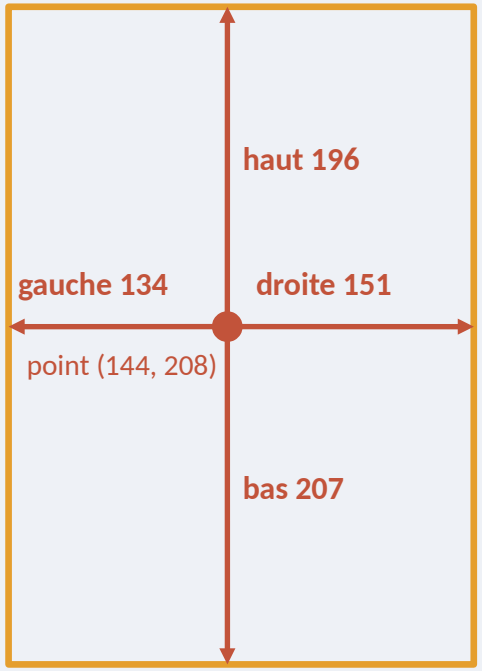
grands objets

À 416 px : 2 704 + 676 + 169 = **3 549 boîtes candidates** par image. Plus de résolution, plus de points : mieux pour les petits objets, plus lent.

Source : Lin et al., Feature Pyramid Networks, CVPR 2017 ; valeurs des pas lues dans yolov8n.pt (Ultralytics 8.4.175).

Ce que prédit chaque point

Pourquoi : la sortie du réseau, avant tout post-traitement.



Boîte : [144 − 134, 208 − 196, 144 + 151, 208 + 207] = [10, 12, 295, 415]

illustration : distances du centre d'une case (pas 32) aux bords de la boîte finale

4 distances aux bords

gauche, haut, droite, bas, depuis le point. Pour chacune, le réseau sort 16 probabilités, une par distance possible (0 à 15 pas de grille) ; la distance retenue est leur moyenne pondérée. Un bord flou peut ainsi rester incertain (perte DFL, partie 4).

Un score par classe

entre 0 et 1, indépendants (une sigmoïde par classe) : 80 scores pour COCO, 12 au TD. Le score de confiance de la boîte est le plus élevé des 80, et sa classe celle de ce score.

Sans boîtes d'ancrage

YOLOv2 à v5 ajustaient des boîtes types fixées à l'avance ; YOLOv8, comme YOLOX en 2021, prédit directement les distances.

Source : Ge et al., YOLOX, 2021 ; Li et al., Generalized Focal Loss, NeurIPS 2020 ; code Ultralytics (tête Detect, reg_max = 16).

YOLO en PyTorch : ce que sort le réseau

Pourquoi : relier le tenseur que sort le réseau aux boîtes que l'on affiche.

```python
import torch
from ultralytics import YOLO

modele = YOLO("yolov8n.pt")          # pré-entraîné sur COCO, 80 classes
reseau = modele.model.eval()         # le réseau PyTorch lui-même

x = torch.zeros(1, 3, 416, 416)      # un lot d'une image 416 x 416
with torch.no_grad():
    sortie = reseau(x)[0]
print(sortie.shape)      # torch.Size([1, 84, 3549]) : 4 + 80 par point
print(52 * 52 + 26 * 26 + 13 * 13)  # 3549 : trois grilles
print(reseau.model[-1].stride)       # tensor([ 8., 16., 32.]) : les pas
```

- On donne au réseau une image **vide** de 416 × 416, juste pour voir la **forme** de sa sortie
- Il sort **(1, 84, 3 549)** : la tête a déjà converti les 4 distances en une boîte (cx, cy, w, h), suivie de 80 scores
- **3 549** = $52^2 + 26^2 + 13^2$: les trois grilles de pas **8, 16 et 32**
- Hors du réseau : seuil de confiance, puis **NMS par classe**
- À l'entraînement, chaque objet est confié à **jusqu'à 10 points** voisins : ils le prédisent tous, d'où les doublons que supprime la NMS

Source : Code Ultralytics 8.4.175 : tête Detect et post-traitement (NMS par classe, max_det = 300).

YOLO en PyTorch : predict, l'appel du TD

Pourquoi : l'interface utilisée tout l'après-midi.

```python
from ultralytics import YOLO

modele = YOLO("yolov8n.pt")
r = modele.predict("astronaute.jpg", imgsz=416, conf=0.25, iou=0.7,
                   verbose=False)[0]

print(r.boxes.xyxy.round())       # [x1, y1, x2, y2], pixels d'origine
# tensor([[ 12.,  14., 364., 511.],
#         [  1.,   7., 507., 512.]])
print(r.boxes.conf.round(decimals=2))   # tensor([0.7500, 0.2700])
print([modele.names[int(c)] for c in r.boxes.cls])
# ['person', 'person']
```

- L'appel du TD : une image en entrée, la liste des détections en sortie. **predict** fait tout : redimensionnement, réseau, seuil **conf**, NMS au seuil **iou**
- **boxes.xyxy**, **.conf**, **.cls** : des tenseurs, en pixels de l'image d'origine
- La seconde (0,27) : une **fausse alerte**. La NMS à 0,5 de la diapositive 14 la supprimait (IoU 0,68) ; celle d'Ultralytics, à 0,7, la garde
- Au TD : **classes=[0]** (COCO), puis **[6]** (Person, fine-tuné) ; un chemin ou une image PIL (NumPy est lu en **BGR**)

Source : Documentation Ultralytics : mode predict et objet Results.

Cinq tailles de YOLOv8 : n, s, m, l, x

Pourquoi : le premier compromis d'un projet : qualité contre temps de calcul.

Cinq tailles : **n** (nano), **s** (small), **m** (medium), **l** (large), **x** (extra-large). C'est la **même architecture**, avec plus ou moins de couches et de canaux : plus gros, c'est plus juste, mais plus lent.

Modèle	Paramètres	Calcul (GFLOPs)	mAP@0.5:0.95 (COCO)	Temps CPU (ms / image)
YOLOv8n	3,2 M	8,7	37,3	80
YOLOv8s	11,2 M	28,6	44,9	128
YOLOv8m	25,9 M	78,9	50,2	235
YOLOv8l	43,7 M	165,1	52,9	375
YOLOv8x	68,2 M	257,8	53,9	479

Paramètres : nombre de poids du réseau. GFLOPs : milliards d'opérations pour une image. mAP@0.5:0.95 : qualité des boîtes sur COCO, en % (définie en partie 3). Temps : une image sur CPU, modèle exporté au format ONNX (partie 4), 640 px.



De n à x : **+16,6 points** de mAP pour **30 fois** plus de calcul. On prend le plus petit modèle qui suffit ; au TD : YOLOv8n.

Source : Ultralytics, page YOLOv8 : mAP sur COCO val2017, vitesse CPU (ONNX) à 640 px.

YOLO : à retenir

1. YOLO est **one-stage** : un backbone CNN, puis une tête qui prédit en chaque point de **trois grilles** (pas de 8, 16 et 32 pixels) ; 3 549 points à 416 px.

2. Chaque point prédit les **distances aux quatre bords** d'une boîte et un **score par classe**, sans boîtes d'ancrage ; un seuil de confiance puis une NMS donnent les détections.

3. Avec Ultralytics, **predict** renvoie les boîtes, scores et classes ; la taille du modèle (n à x) est un compromis entre mAP et temps de calcul.

Dans le TD : Étape 4 : YOLOv8n pré-entraîné sur COCO face aux images de chantier ; étape 6 : son fine-tuning.

Pour approfondir : Documentation Ultralytics : YOLOv8 ; Redmon et al., 2016 : l'article YOLO (arXiv)



Partie 3

Mesurer un détecteur

Pourquoi : comparer YOLO à une baseline et choisir un seuil, aux étapes 5 et 7 du TD

Évaluer un détecteur : définition

Définition

Rappel : un détecteur rend, pour chaque objet trouvé, une boîte, une classe et un score. L'évaluer, c'est compter sur des images annotées à la main ce qu'il trouve, ce qu'il invente et ce qu'il rate, puis résumer ces comptes en quelques chiffres : précision, rappel, AP, mAP.

Le problème que cela résout

Un détecteur peut trouver tous les casques en déclenchant sans arrêt, ou ne jamais se tromper en ne signalant presque rien. Il faut des mesures qui voient les deux, et un seuil d'alerte choisi selon ce que coûte chaque erreur.

Ce qui entre, ce qui sort

Entrée : les détections du modèle (boîte, classe, score) et les boîtes annotées. Sortie : précision et rappel pour un seuil donné ; AP et mAP, qui résument tous les seuils.

Pour en savoir plus : Ultralytics : YOLO Performance Metrics ; COCO : Detection Evaluation

Ce que l'on va voir, dans l'ordre

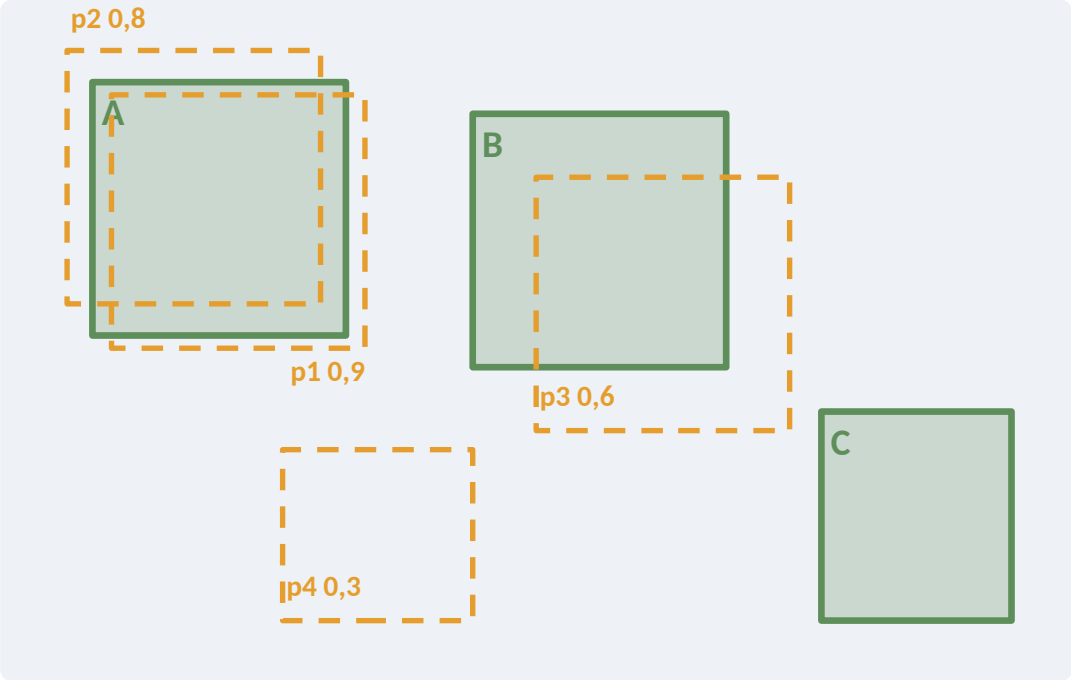
2. Vrais positifs, fausses alertes, oublis
3. Le seuil de confiance : précision contre rappel
4. L'AP : résumer tous les seuils en un chiffre
5. mAP@0.5 ou mAP@0.5:0.95 : quelle exigence ?
6. Mesurer le mAP avec Ultralytics
7. Une baseline sans apprentissage
8. Choisir le seuil selon le coût des erreurs
9. À retenir

Dans le TD : Étape 5 : votre évaluation (TODO 4) et la baseline couleur ; étape 7 : précision et rappel selon le seuil (TODO 5).

Vrais positifs, fausses alertes, oublis

Pourquoi : c'est la règle qui transforme des boîtes en vrais positifs, fausses alertes et oublis.

On compare les boîtes du modèle aux boîtes **annotées** à la main. Une prédiction qui tombe sur un objet (IoU ≥ 0,5) est un **vrai positif** ; les autres sont des **fausses alertes** ; un objet que rien ne trouve est un **oubli**.



vert : objets annotés ; pointillés : prédictions, avec leur score

- p1 (0,9) recouvre A, IoU 0,78 : **vrai positif**
- p2 (0,8) recouvre aussi A, déjà trouvé : **fausse alerte** (doublon)
- p3 (0,6) recouvre mal B, IoU 0,39 : **fausse alerte**
- p4 (0,3) ne recouvre rien : **fausse alerte**
- B et C, trouvés par aucune : **2 oublis**

Précision = 1 / 4 = 0,25
parmi les alertes, la part de justes

Rappel = 1 / 3 = 0,33
parmi les objets, la part trouvée

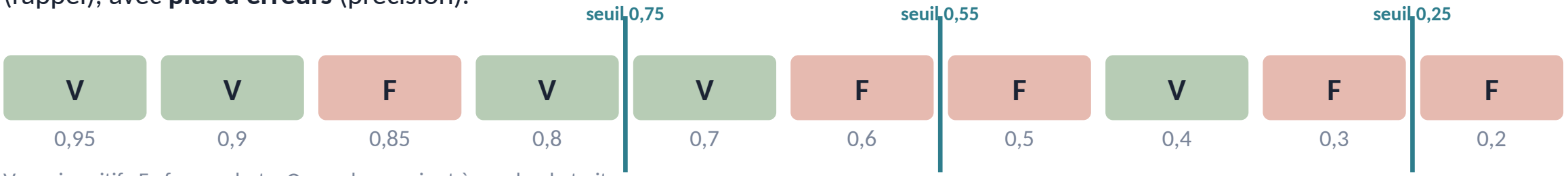
Ordre : par **score décroissant**, chaque prédiction prend l'objet libre de plus grande IoU. Pas de « vrais négatifs » : on ne compte pas le fond.

Source : Lin et al., Microsoft COCO, ECCV 2014 ; règle d'appariement de l'évaluation COCO.

Le seuil de confiance : précision contre rappel

Pourquoi : chaque seuil de confiance donne un compromis différent entre fausses alertes et oublis.

Chaque détection a un **score** ; le **seuil** dit à partir de quel score on la garde. Ici, 10 détections d'une classe, triées par score, et 6 objets à trouver (exemple repris dans les trois diapositives suivantes). Baisser le seuil, c'est garder plus de détections : on trouve **plus d'objets** (rappel), avec **plus d'erreurs** (précision).



V : vrai positif ; F : fausse alerte. On garde ce qui est à gauche du trait.

Seuil	Gardées	Vrais positifs	Précision (justes / gardées)	Rappel (trouvés / 6)	F1
0,75	4	3	0,75	0,50	0,60
0,55	6	4	0,67	0,67	0,67
0,25	9	5	0,56	0,83	0,67

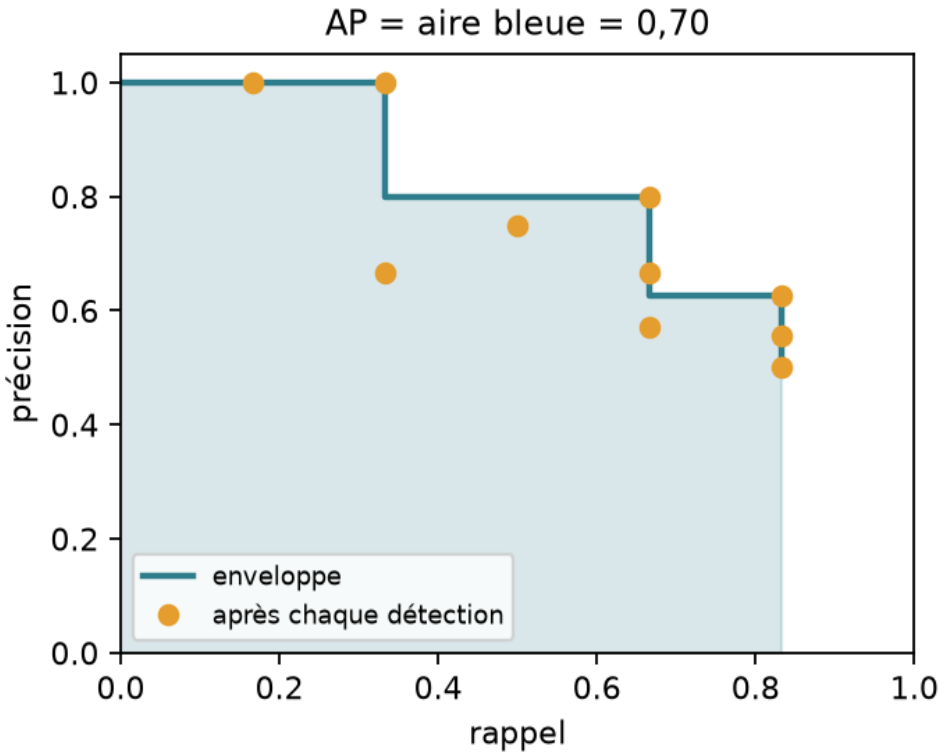
Baisser le seuil : **plus de rappel, moins de précision**. Le **F1** = 2PR / (P + R) les résume ; il n'est élevé que si les deux le sont.

Source : Exemple construit pour le cours ; même calcul que le TODO 5 du TD.

L'AP : résumer tous les seuils en un chiffre

Pourquoi : résumer la qualité d'un détecteur sur tous les seuils à la fois.

Mêmes 10 détections et 6 objets. Plutôt que de choisir un seuil, on les essaie **tous** : chaque seuil donne un point (rappel, précision). L'**AP** (**Average Precision**, précision moyenne) est l'**aire sous la courbe** : 1 si le détecteur est parfait, près de 0 s'il ne trouve rien. Le **mAP** (**mean AP**) est la moyenne des AP de toutes les classes.



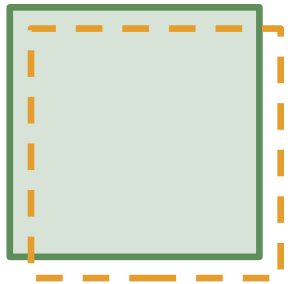
- **Ronds orange** : précision et rappel après chaque détection, du score le plus haut au plus bas
- **Courbe bleue** : pour chaque rappel, la meilleure précision possible à ce rappel ou au-delà
- **AP** = aire bleue = (1 + 1 + 0,8 + 0,8 + 0,625) × 1/6 = **0,70**
- Le 6e objet n'est jamais trouvé : rien au-delà d'un rappel de 0,83
- **mAP** : la moyenne des AP de toutes les classes, le chiffre affiché par Ultralytics

Source : Everingham et al., IJCV 2010 (enveloppe de la courbe ; aire complète dans le devkit VOC depuis 2010) ; Ultralytics interpole sur 101 points, comme COCO.

mAP@0.5 ou mAP@0.5:0.95 : quelle exigence ?

Pourquoi : les deux chiffres affichés par Ultralytics, et ce qui les sépare.

Le nombre après le @ est le **seuil d'IoU** à partir duquel une boîte compte comme juste. **mAP@0.5** : il suffit d'une IoU d'au moins 0,5, un recouvrement correct sans plus. **mAP@0.5:0.95** : on refait le calcul avec **10 exigences**, de 0,5 à 0,95, et on fait la moyenne : il faut des boîtes bien placées.



IoU 0,72

Cette boîte, jugée à chacun des dix seuils :

0,5	0,55	0,6	0,65	0,7	0,75	0,8	0,85	0,9	0,95
juste	**juste**	**juste**	**juste**	**juste**	**fausse**	**fausse**	**fausse**	**fausse**	**fausse**

Juste 5 fois sur 10 : elle compte **pleinement** dans le mAP@0.5, **à moitié** dans le mAP@0.5:0.95.

Petit objet, 20 × 20 px, boîte décalée de 4 px en x et en y

IoU 0,47 : déjà fausse au seuil 0,5

Grand objet, 100 × 100 px, même décalage

IoU 0,85 : juste jusqu'au seuil 0,85

mAP@0.5:0.95 ≤ mAP@0.5, toujours ; l'écart grandit pour les **petits** objets, où quelques pixels d'erreur coûtent beaucoup d'IoU.

Source : Lin et al., Microsoft COCO, ECCV 2014 : moyenne sur les seuils d'IoU 0,50 à 0,95.

Mesurer le mAP avec Ultralytics

Pourquoi : obtenir ces chiffres en une ligne, comme à l'étape 6.

```python
from ultralytics import YOLO

modele = YOLO("tp2_yolov8n_ppe_20ep.pt")   # fourni : déjà fine-tuné, 20 époques
res = modele.val(data="construction-ppe/ppe.yaml", split="val",
                 imgsz=416, batch=16, workers=0, plots=False,
                 verbose=False)

print(round(res.box.map50, 3), round(res.box.map, 3))   # 0.668 0.445
for k, c in enumerate(res.box.ap_class_index[:3]):   # AP par classe
    print(modele.names[int(c)], round(float(res.box.ap50[k]), 3),
          round(float(res.box.ap[k]), 3))
# helmet 0.734 0.535
# gloves 0.691 0.459
# vest 0.837 0.601
```

- **val** calcule tout : appariement, courbes, AP par classe, mAP
- **split="val"** : on mesure sur la validation tant que l'on compare et règle
- **box.map50**, **box.map** : mAP@0.5 et mAP@0.5:0.95 ; **box.ap50**, **box.ap** : par classe

Source : Documentation Ultralytics : mode val (box.map, box.map50) ; ap et ap50 par classe : code ultralytics/utils/metrics.py.

Une baseline sans apprentissage

Pourquoi : comme au Jour 1 : savoir si le deep learning vaut son coût.

Une **baseline** est une méthode simple, **sans apprentissage**, qui sert de point de comparaison : si YOLO ne fait pas mieux, il ne vaut pas son coût. Ici : repérer les casques jaunes et orange par leur **couleur**.

Image RGB	HSV	inRange	Ouverture	Composantes	Règles de forme
l'image d'entrée	teinte (la couleur), saturation (vivacité), valeur (luminosité)	teinte 15 à 30 sur 179 (OpenCV) : de l'orange au jaune, vive et claire	érosion puis dilatation : efface les petits amas	une boîte par région d'au moins aire_min pixels (TODO 3)	carrée, < 3 % de l'image

casques annotés



masque couleur (après ouverture)



baseline : 2 boîtes



- Toutes les boîtes ont le **même score** : pas de seuil de confiance, pas de courbe précision-rappel
- Sur cette image : les deux casques **jaunes** sont trouvés ; mais la plupart des casques du jeu sont **blancs**, bleus ou verts, hors de la plage
- Et l'**absence** de casque (l'homme de gauche), une règle de couleur ne peut pas la voir
- Réglée sur la **validation**, comme YOLO

Une image de validation de Construction-PPE, réglages par défaut du TD (teinte 15 à 30, saturation ≥ 100, valeur ≥ 175, aire ≥ 900 px).

Source : Documentation OpenCV : Changing Colorspaces (cvtColor vers HSV, inRange) ; figure calculée pour le cours.

Choisir le seuil selon le coût des erreurs

Pourquoi : en production, il faut un seul seuil, et c'est une décision métier.

En production, il faut **un seul** seuil : à partir de quel score déclenche-t-on une alerte ? On le choisit selon ce que **coûte** chaque erreur. Exemple : un oubli (une personne sans casque non signalée) coûte 10, une fausse alerte (un déplacement pour rien) coûte 1.

Seuil	Oublis	Fausses alertes	Coût (oubli 10, fausse alerte 1)	F1
0,75	3	1	3 × 10 + 1 = 31	0,60
0,55	2	2	2 × 10 + 2 = 22	0,67
0,25	1	4	1 × 10 + 4 = 14	0,67

Ici : 0,25

le coût le plus bas, alors que les F1 de 0,55 et 0,25 sont égaux

Sur la validation

le seuil est un réglage : il se choisit sur la validation, jamais sur le test

Mesuré une fois sur le test

pour chaque classe analysée, avec le seuil figé

Source : Exemple construit pour le cours, avec des coûts choisis pour l'illustration.

Évaluer un détecteur : à retenir

1. On apparie chaque prédiction, par **score décroissant**, à l'objet libre de plus grande IoU : vrai positif si l'IoU atteint le seuil, sinon fausse alerte ; les objets restants sont des oublis. Pas de vrais négatifs.

2. L'**AP** est l'aire sous la courbe précision-rappel d'une classe ; le **mAP** est leur moyenne. Le mAP@0.5:0.95 exige en plus des boîtes bien placées.

3. En production, on choisit **un seul seuil**, sur la validation, selon le coût d'un oubli et d'une fausse alerte ; on le mesure ensuite une fois sur le test.

Dans le TD : Étape 5 : votre évaluation (TODO 4) et la baseline couleur ; étape 7 : précision et rappel selon le seuil (TODO 5).

Pour approfondir : Ultralytics : YOLO Performance Metrics ; COCO : Detection Evaluation



Après la partie 3

Pause

Reprise avec le fine-tuning de YOLO. Pendant la pause, vérifiez que uv run python verifier_installation.py --donnees a bien téléchargé Construction-PPE (environ 650 Mo)



Partie 4

Fine-tuner YOLO

Pourquoi : apprendre nos 12 classes à un détecteur pré-entraîné sur COCO, à l'étape 6 du TD

Fine-tuner YOLO : définition

Définition

Fine-tuner, c'est partir d'un YOLO déjà entraîné sur COCO (80 classes d'objets courants) et le réentraîner quelques époques sur nos images de chantier, pour qu'il apprenne nos 12 classes. C'est le transfer learning du Jour 1, appliqué à un détecteur.

Le problème que cela résout

Le YOLO de COCO connaît « personne », mais ni « casque » ni « absence de casque ». Tout apprendre de zéro demanderait beaucoup plus d'images et de calcul que nos 700 images et un CPU : on réutilise ce qu'il sait déjà voir.

Ce qui entre, ce qui sort

Entrée : yolov8n.pt (les poids appris sur COCO) et nos images annotées (une image, un fichier texte de boîtes). Sortie : un nouveau fichier de poids, best.pt, qui détecte nos 12 classes, et son mAP sur la validation.

Pour en savoir plus : Ultralytics : Train ; Ultralytics : Export

Ce que l'on va voir, dans l'ordre

2. Ce qui change dans le réseau

3. Le format des données : une image, un fichier texte

4. Les trois pertes de YOLO

5. Fine-tuner en code

6. Exporter le modèle

7. À retenir

Dans le TD : Étape 6 : un fine-tuning court sur votre machine (environ 2 min 30 par époque sur CPU), puis l'analyse du modèle fourni (20 époques).

Ce qui change dans le réseau

Pourquoi : comprendre ce que fait le fine-tuning de l'étape 6, au-delà de l'appel à train().

Backbone	repris de COCO
Cou (neck)	repris de COCO
Tête : branche des boîtes	reprise de COCO
Tête : branche des classes	**dernières couches 80 → 12 sorties**

Puis tout le réseau est réentraîné (aucun étage gelé par défaut).

- Comme au Jour 1 : on garde ce qui est **générique** (bords, textures, formes), on remplace ce qui est **propre aux classes**
- Les classes de **même nom** (person → Person) gardent leur ligne de poids COCO
- Journal : « Remapped **1/12** cls head rows » : la ligne Person reprend celle de person ; les 11 autres partent d'une initialisation neuve (le « Transferred 355/355 » compte cette couche)

Source : Code Ultralytics 8.4.175 : BaseModel.load et _remap_cls_by_names ; journal d'entraînement.

Le format des données : une image, un fichier texte

Pourquoi : c'est le format que lit Ultralytics, et celui de la brique 2 du projet.

Pour entraîner YOLO, il faut chaque image et, à côté, un **fichier texte du même nom** qui liste ses boîtes, **une par ligne**. Un fichier de configuration (**.yaml**) dit où sont les images et comment s'appellent les classes.



labels/train/source-40471b5afe759eb420a90db737aa9a93.txt

```
0 0.397480 0.173360 0.345020 0.346670   helmet
2 0.406500 0.689270 0.552710 0.621460   vest
4 0.438000 0.416000 0.284000 0.214000   goggles
5 0.454110 0.500010 0.662350 0.999980   Person
8 0.747000 0.984000 0.072000 0.032000   no_gloves
```

classe, puis centre x, centre y, largeur, hauteur, divisés par la taille de l'image (640 × 480) ; le nom de la classe est ajouté ici pour la lecture

ppe.yaml (le fichier de configuration, souvent nommé data.yaml)

```
path: construction-ppe     # images/ et labels/, chacun en train/ val/ test/
train: images/train
val: images/val
names: {0: helmet, ..., 5: Person, 6: no_helmet, ..., 11: no_vest}
```

D'où viennent ces fichiers ? Au TD, ils sont **fournis** avec Construction-PPE : 10 429 boîtes pour l'entraînement, tracées ou vérifiées par des annotateurs. Pour vos propres images, on les trace avec un **outil d'annotation** (makesense.ai, CVAT, Label Studio), qui exporte ce format ; pour NEU-DET, **tp.voc_vers_yolo** les écrit à partir des .xml.

Source : Documentation Ultralytics : format des jeux de détection.

Les trois pertes de YOLO

Pourquoi : savoir ce que l'entraînement optimise, et lire ses pertes (barre de progression, results.csv).

Une **perte** est un nombre qui mesure l'erreur du modèle sur un lot d'images ; l'entraînement ajuste les poids pour la faire **baisser**. YOLO en additionne **trois**, chacune avec un poids.

`box_loss`	× 7,5	**La boîte est-elle bien placée ?** 1 − IoU (une IoU améliorée, la CIoU, Complete IoU, qui regarde aussi les centres et la forme)
`cls_loss`	× 0,5	**La classe est-elle la bonne ?** entropie croisée, comme au Jour 1, en version binaire : une par classe
`dfl_loss`	× 1,5	**Les bords sont-ils précis ?** DFL (Distribution Focal Loss) : resserre chaque distribution de distance sur la bonne valeur

Au TD : les trois pertes s'affichent dans la barre de progression et dans **results.csv**. Elles doivent **baisser** ; si celles de la validation remontent, c'est du **surapprentissage**.

Source : Zheng et al., Distance-IoU Loss, AAAI 2020 (CIoU) ; Li et al., Generalized Focal Loss, 2020 (DFL) ; gains par défaut d'Ultralytics.

Fine-tuner en code

Pourquoi : l'étape 6 du TD, en quelques lignes.

```python
from ultralytics import YOLO

modele = YOLO("yolov8n.pt")          # départ : COCO, tête à 80 classes
modele.train(data="construction-ppe/ppe.yaml",  # images et 12 classes
             epochs=1, imgsz=320, fraction=0.1,  # court, pour l'exemple
             batch=16, workers=0, plots=False, verbose=False,
             project="sorties", name="essai", exist_ok=True)

print(modele.model.model[-1].nc)    # 12 : la nouvelle tête
print(modele.names)
# {0: 'helmet', 1: 'gloves', 2: 'vest', 3: 'boots', 4: 'goggles',
#  5: 'Person', 6: 'no_helmet', 7: 'no_goggle', ...}
```

- **YOLO("yolov8n.pt")** : le départ, les poids COCO
- **train** lit le ppe.yaml, refait la tête à 12 classes, entraîne, valide ; poids dans …/weights/**best.pt**
- **epochs**, **imgsz**, **fraction** : les leviers du coût ; **batch=16**, **workers=0** sur CPU
- Coût : environ **2 min 30 par époque** sur CPU (modèle fourni : 20 époques, 50 min) ; temps ≈ images × époques × résolution²

Source : Documentation Ultralytics : mode train.

Exporter le modèle

Pourquoi : un modèle entraîné doit sortir du notebook pour être utilisé.

```python
from ultralytics import YOLO

modele = YOLO("tp2_yolov8n_ppe_20ep.pt")
chemin = modele.export(format="torchscript", imgsz=416)   # un fichier
print(chemin)                          # tp2_yolov8n_ppe_20ep.torchscript

exporte = YOLO(chemin)                 # s'utilise comme l'original
r = exporte.predict("astronaute.jpg", imgsz=416, conf=0.3,
                    verbose=False)[0]
print([exporte.names[int(c)] for c in r.boxes.cls])
# ['Person']
```

- **export** produit un fichier autonome : TorchScript ici ; ONNX, TensorRT, OpenVINO : formats lus par d'autres moteurs (CPU, GPU NVIDIA, puces Intel)
- Le modèle exporté se recharge avec **YOLO(chemin)** et s'utilise comme l'original
- Sur l'astronaute, au seuil 0,3, il ne garde qu'une **personne** ; son meilleur « gilet » (la combinaison orange) a un score de **0,29**, juste en dessous
- Quantification, mesure sur la machine cible : Jour 6

Source : Documentation Ultralytics : mode export.

Fine-tuner YOLO : à retenir

1. Le fine-tuning garde le backbone et le cou du réseau ; seules les **couches de sortie des classes** changent de taille (80 → 12), et **tout** le réseau est réentraîné.

2. Les données : une image, un fichier **.txt** (une ligne « classe cx cy w h » par boîte) et un fichier de configuration **.yaml** ; la perte additionne un terme de boîte, un de classe et un de distribution (DFL).

3. Le coût réel : environ **2 min 30 par époque** sur CPU pour ce jeu, et le temps d'**annotation** ; le modèle s'**exporte** ensuite hors du notebook.

Dans le TD : Étape 6 : un fine-tuning court sur votre machine (environ 2 min 30 par époque sur CPU), puis l'analyse du modèle fourni (20 époques).

Pour approfondir : Ultralytics : Train ; Ultralytics : Export



Partie 5

Faster R-CNN, l'approche en deux étapes

Pourquoi : l'autre grande famille de détecteurs, et la base de Mask R-CNN pour la segmentation

Faster R-CNN : définition

Définition

Faster R-CNN (R-CNN : Region-based CNN, troisième version après R-CNN et Fast R-CNN) est un détecteur en deux étapes. D'abord un petit réseau, le RPN (Region Proposal Network, réseau de propositions de régions), repère les zones qui pourraient contenir un objet, sans dire lequel. Puis une tête examine chaque zone : quelle classe, et quelle boîte exacte.

Le problème que cela résout

Séparer « où pourrait-il y avoir un objet ? » de « lequel, exactement ? » : la tête ne travaille que sur un nombre limité de régions (jusqu'à 1 000 par image en inférence dans torchvision). C'est aussi la base de Mask R-CNN.

Ce qui entre, ce qui sort

Entrée : une liste d'images. Sortie : un dictionnaire par image (boxes, labels, scores) ; en entraînement, quatre pertes.

Ce que l'on va voir, dans l'ordre

2. Deux étapes : proposer, puis classer
3. Faster R-CNN en PyTorch
4. One-stage ou two-stage ?
5. À retenir

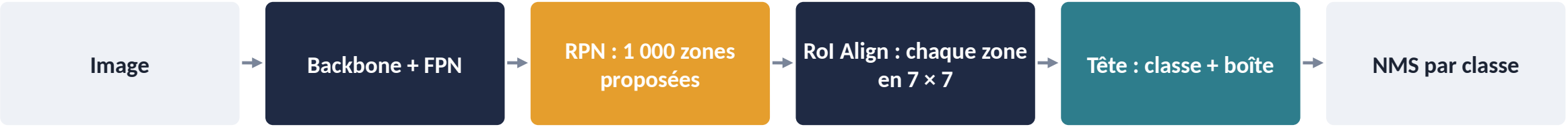
Dans le TD : Pour aller plus loin, en fin de TD : comparer sa vitesse à celle de YOLOv8n.

Pour en savoir plus : Tutoriel torchvision : Object Detection Finetuning ; Ren et al., 2015 : l'article Faster R-CNN (arXiv)

Deux étapes : proposer, puis classer

Pourquoi : l'autre grande famille de détecteurs, et la base de Mask R-CNN.

Faster R-CNN travaille en **deux temps** : d'abord « **où** pourrait-il y avoir un objet ? », puis, zone par zone, « **lequel** est-ce, et où sont ses bords exactement ? ».



1. Le RPN : proposer des zones

RPN = Region Proposal Network, réseau de propositions de régions. Un petit réseau parcourt les cartes de features ; en chaque point, il teste des boîtes de plusieurs tailles et formes (les **boîtes d'ancrage**) et donne à chacune un score « objet ou fond », **sans dire quel objet**. On garde les 1 000 meilleures zones par image (torchvision, en inférence).

2. Examiner chaque zone

RoI Align (RoI : Region of Interest) découpe dans les cartes de features la zone proposée et la ramène à une grille fixe de **7 × 7**, quelle que soit sa taille. La **tête** dit quelle classe c'est (nos classes, ou « fond ») et ajuste la boîte.

La tête travaille sur jusqu'à 1 000 zones par image : plus précis à son époque, plus lent.

Source : Ren et al., Faster R-CNN, NeurIPS 2015 ; Lin et al., Feature Pyramid Networks, 2017 ; réglages par défaut de torchvision.

Faster R-CNN en PyTorch

Pourquoi : le même travail qu'avec YOLO, avec les outils de torchvision.

```python
import torch
from torchvision.models.detection import fasterrcnn_mobilenet_v3_large_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

modele = fasterrcnn_mobilenet_v3_large_fpn(weights="DEFAULT")   # COCO
tete = modele.roi_heads.box_predictor
modele.roi_heads.box_predictor = FastRCNNPredictor(
    tete.cls_score.in_features, 13)        # 12 classes + le fond

image = torch.rand(3, 416, 416)            # (C, H, W), valeurs 0 à 1
modele.eval()
with torch.no_grad():
    sortie = modele([image])[0]            # une liste d'images en entrée
print(list(sortie))     # ['boxes', 'labels', 'scores']

modele.train()                             # on donne aussi les vérités
cible = {"boxes": torch.tensor([[50., 60., 200., 300.]]),
         "labels": torch.tensor([7])}
print(list(modele([image], [cible])))      # 2 pertes tête, 2 pertes RPN
# ['loss_classifier', 'loss_box_reg', 'loss_objectness', 'loss_rpn_box_reg']
```

- Comme au Jour 1 : charger les poids COCO, puis **remplacer la tête** (FastRCNNPredictor, 12 classes + le fond)
- En évaluation : une **liste d'images**, un **dictionnaire** par image (boxes, labels, scores)
- En entraînement : on donne aussi les vérités, et le modèle rend **quatre pertes**
- Au TD (pour aller plus loin) : comparer sa vitesse à celle de YOLOv8n

Source : Tutoriel torchvision : TorchVision Object Detection Finetuning.

One-stage ou two-stage ?

Pourquoi : savoir lequel choisir, chiffres à l'appui.

One-stage (YOLO) : **une seule passe** du réseau donne directement boîtes, classes et scores. **Two-stage** (Faster R-CNN) : une passe **propose des zones**, une seconde **examine chacune**. Le premier est plus rapide ; le second a longtemps été plus précis.

Modèle	Famille	mAP@0.5:0.95 (COCO)	Calcul (GFLOPs)
Faster R-CNN MobileNetV3-FPN	two-stage	32,8	4,5
Faster R-CNN ResNet-50 FPN	two-stage	37,0	134
Faster R-CNN ResNet-50 FPN v2	two-stage	46,7	280
YOLOv8n (640 px)	one-stage	37,3	8,7
YOLOv8m (640 px)	one-stage	50,2	78,9



À mAP comparable (environ 37), YOLOv8n demande **15 fois moins** de calcul que Faster R-CNN ResNet-50 ; Faster R-CNN MobileNetV3, plus léger, perd 4,2 points sur le ResNet-50. Le two-stage reste surtout utile comme base de **Mask R-CNN**, et parce qu'il est prêt à l'emploi dans torchvision.

Source : torchvision (métadonnées des poids COCO) et Ultralytics (page YOLOv8) : mesures faites dans des conditions différentes.

Faster R-CNN : à retenir

1. Étape 1 : le **RPN** propose des régions (un score « objet » et un ajustement par boîte d'ancrage) ; étape 2 : **RoI Align** en extrait une carte 7 × 7, la tête la classe et affine la boîte.

2. Dans torchvision, on remplace la tête par **FastRCNNPredictor** (nos classes + le fond) ; le modèle rend des dictionnaires, et quatre pertes en entraînement.

3. Aujourd'hui, un YOLO récent égale ou dépasse Faster R-CNN sur COCO pour bien moins de calcul ; le two-stage reste surtout utile comme base de Mask R-CNN, et parce qu'il est prêt à l'emploi dans torchvision.

Dans le TD : Pour aller plus loin, en fin de TD : comparer sa vitesse à celle de YOLOv8n.

Pour approfondir : Tutoriel torchvision : Object Detection Finetuning ; Ren et al., 2015 : l'article Faster R-CNN (arXiv)



Partie 6

Segmentation : U-Net, DeepLab et SAM

Pourquoi : quand une boîte ne suffit pas, on veut la forme exacte ; c'est l'étape 8 du TD

TD · étape 8

La segmentation : une classe pour chaque pixel

Pourquoi : savoir ce qu'apporte un masque par rapport à une boîte.

Segmenter une image, c'est dire, pour **chaque pixel**, à quoi il appartient. Le résultat est un **masque** : une image de même taille où chaque pixel porte une classe. Là où la détection donne une boîte, qui englobe aussi du fond, la segmentation donne la **forme exacte**.



Classification

une étiquette pour l'image



Détection

une boîte : 67 % de l'image



Segmentation

un masque : 41 % de l'image

Source : Image « astronaut » de scikit-image ; YOLOv8n (COCO, 416 px) et MobileSAM, calculés pour le cours.

TD · étape 8

Trois types de segmentation

Pourquoi : savoir situer chaque besoin et chaque modèle.



Masques de SAM (Segment Anything Model, présenté plus loin) à partir des boîtes « Person » du modèle fine-tuné ; au centre, fusionnés en une seule classe.

Sémantique

une classe par pixel ; toutes les personnes forment une seule région

Modèles : U-Net, DeepLab

Usage : surface d'un défaut, zone libre, organe

D'instances

un masque par objet : chaque personne séparément

Modèles : Mask R-CNN ; YOLO + SAM

Usage : compter, mesurer chaque objet

Panoptique

les deux : une classe par pixel, et un numéro par objet

Modèles : Mask2Former, Panoptic FPN (pour mémoire)

Usage : scènes complètes (conduite autonome)

Source : Kirillov et al., Panoptic Segmentation, CVPR 2019.

La segmentation sémantique : définition

Définition

La segmentation sémantique attribue une classe à chaque pixel de l'image. Toutes les personnes forment une seule région « personne » : on ne distingue pas les individus.

Le problème que cela résout

Une boîte englobe aussi du fond ; certaines tâches veulent la forme exacte (surface d'un défaut, zone libre, organe). On la mesure par le mIoU.

Ce qui entre, ce qui sort

Entrée : une image. Sortie : un score par classe et par pixel (N, C, H, W), puis une carte de classes (N, H, W).

Ce que l'on va voir, dans l'ordre

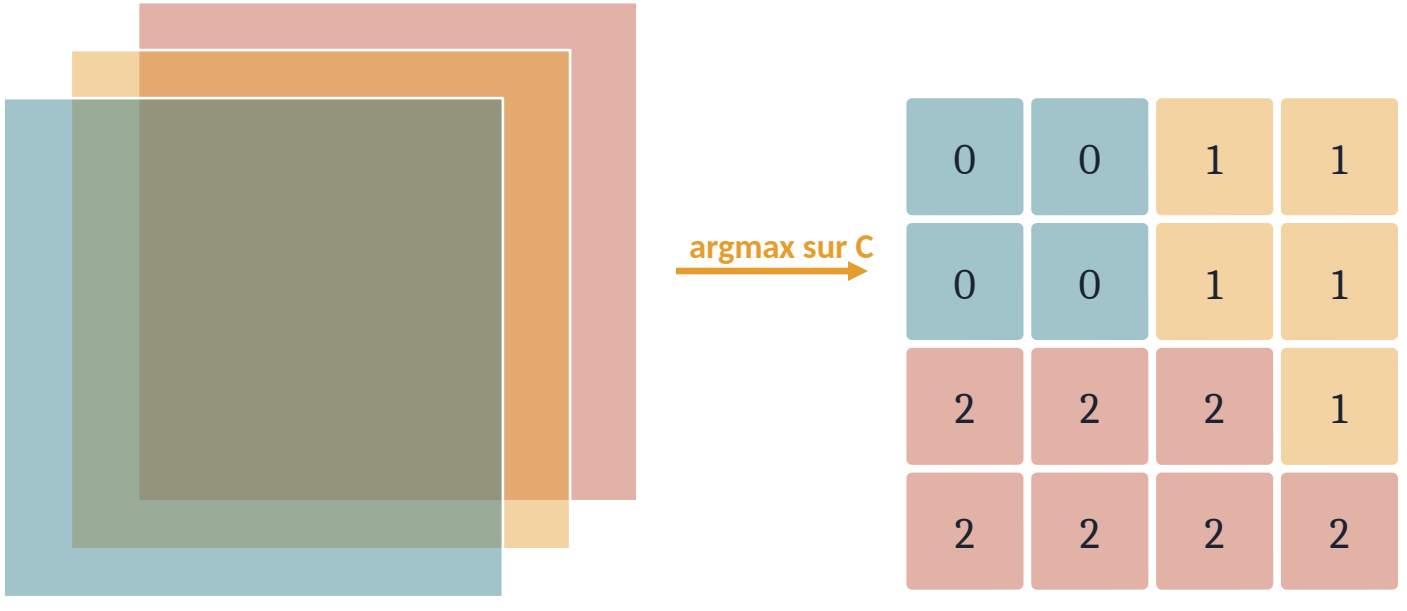
2. Une classification par pixel
3. Le mIoU (mean IoU) : mesurer une segmentation
4. mIoU en PyTorch
5. U-Net
6. U-Net en PyTorch
7. La convolution dilatée
8. Convolution dilatée en PyTorch
9. DeepLab et son module ASPP
10. DeepLab en PyTorch
11. À retenir

Dans le TD : Étape 8 : DeepLab sur les images de chantier, classe 15 (personne).

Pour en savoir plus : d2l, section 14.9 : Semantic Segmentation and the Dataset ; Ronneberger et al., 2015 : l'article U-Net (arXiv)

Une classification par pixel

Pourquoi : c'est le principe de toute segmentation sémantique.



scores (N, C, H, W)
une carte par classe

classes (N, H, W)
une classe par pixel

- Un réseau **entièrement convolutif** : la sortie a la taille de l'image
- Perte : l'**entropie croisée** du Jour 1, calculée pour chaque pixel puis moyennée
- DeepLab (torchvision) : **21** cartes, 20 classes de Pascal VOC + le fond ; classe **15** : personne

Source : Long, Shelhamer et Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015.

Le mIoU (mean IoU) : mesurer une segmentation

Pourquoi : la mesure de la segmentation, l'équivalent du mAP de la détection.

Le **mIoU** (**mean IoU**, IoU moyenne) reprend l'IoU des boîtes, mais sur des **pixels** : pour chaque classe, les pixels communs à la prédiction et à l'annotation, divisés par les pixels de l'une **ou** de l'autre. Puis on fait la **moyenne** sur les classes.

prédit

0	0	1	1
0	0	1	1
2	2	2	1
2	2	2	2

annoté

0	0	0	1
0	0	1	1
2	2	1	1
2	2	2	2

Classe	En commun	Union	IoU
bleue (0)	4	5	0,80
orange (1)	4	6	0,67
rouge (2)	6	7	0,86

$$\text{mIoU} = (4/5 + 4/6 + 6/7) \,/\, 3 = 0{,}77$$

Le mIoU donne le **même poids à chaque classe** : une grande classe, comme le fond, ne masque pas les petits objets ratés, à la différence de l'exactitude par pixel.

Source : Everingham et al., IJCV 2010 (IoU de segmentation) ; exemple calculé avec PyTorch ; DeepLabV3 MobileNetV3 de torchvision : mIoU 60,3 (COCO val2017, étiquettes VOC).

mIoU en PyTorch

Pourquoi : passer d'un tenseur de scores à une carte de classes, puis au mIoU.

```python
import torch

logits = torch.randn(1, 3, 4, 4)       # (N, C, H, W) : score par pixel
print(logits.argmax(dim=1).shape)      # torch.Size([1, 4, 4])

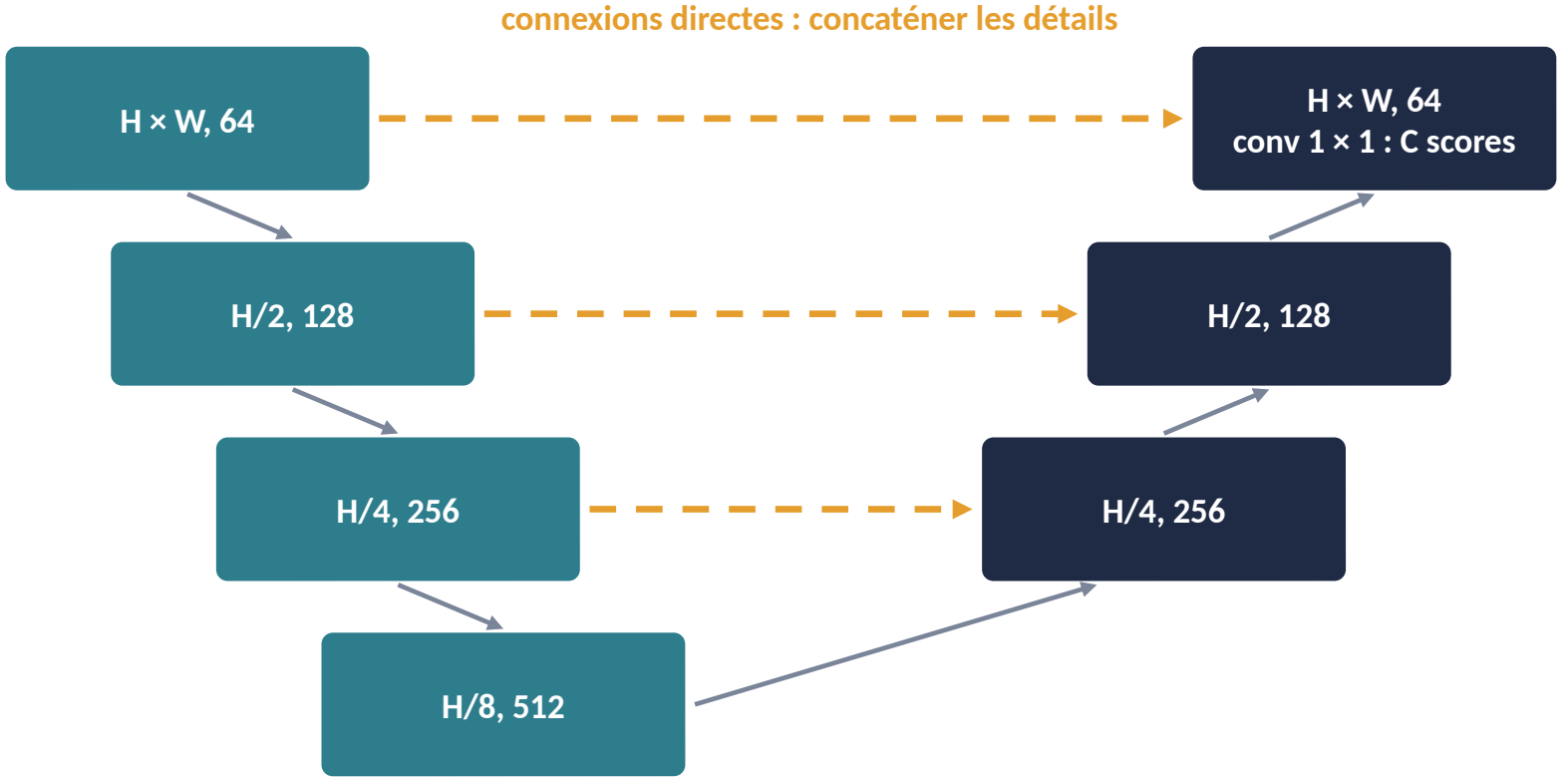
predite = torch.tensor([[0, 0, 1, 1], [0, 0, 1, 1],
                        [2, 2, 2, 1], [2, 2, 2, 2]])
verite = torch.tensor([[0, 0, 0, 1], [0, 0, 1, 1],
                       [2, 2, 1, 1], [2, 2, 2, 2]])
iou = []
for c in range(3):
    inter = ((predite == c) & (verite == c)).sum().item()
    union = ((predite == c) | (verite == c)).sum().item()
    iou.append(inter / union)
print([round(v, 2) for v in iou])     # [0.8, 0.67, 0.86] : IoU par classe
print(round(sum(iou) / 3, 2))         # 0.77 : le mIoU
```

- **argmax(dim=1)** : de (N, C, H, W) à (N, H, W), une classe par pixel
- Pour chaque classe : deux masques booléens, **&** pour l'intersection, **|** pour l'union
- On arrondit **à la fin** : 0,77, et non 0,78 avec les IoU déjà arrondies

Source : Exemple calculé avec PyTorch (code/11_miou.py).

U-Net

Pourquoi : l'architecture de référence quand les données annotées sont rares.



- **Encodeur** : un CNN, le contexte augmente, la position exacte se perd
- **Décodeur** : remonte jusqu'à la résolution de départ
- **Connexions directes** : on concatène les cartes de l'encodeur, au même niveau ; elles rendent les **contours fins**
- Né en **imagerie médicale**, efficace avec peu d'images annotées

Dans chaque bloc : la résolution (H/2 = moitié de la hauteur et de la largeur), puis le nombre de canaux.

Source : Ronneberger, Fischer et Brox, U-Net, MICCAI 2015.

U-Net en PyTorch

Pourquoi : voir où passent l'encodeur, le décodeur et les connexions directes.

```python
import torch
import torch.nn as nn

def bloc(c_in, c_out):
    return nn.Sequential(nn.Conv2d(c_in, c_out, 3, padding=1), nn.ReLU(),
                         nn.Conv2d(c_out, c_out, 3, padding=1), nn.ReLU())

class MiniUNet(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        self.enc1, self.enc2 = bloc(3, 16), bloc(16, 32)    # encodeur
        self.pool = nn.MaxPool2d(2)
        self.monte = nn.ConvTranspose2d(32, 16, 2, stride=2)  # x 2
        self.dec1 = bloc(32, 16)              # 16 remontés + 16 copiés
        self.tete = nn.Conv2d(16, n_classes, 1)   # score par pixel

    def forward(self, x):
        e1 = self.enc1(x)                     # (N, 16, H, W) : détails
        e2 = self.enc2(self.pool(e1))         # (N, 32, H/2, W/2)
        d1 = self.monte(e2)                   # (N, 16, H, W)
        d1 = self.dec1(torch.cat([d1, e1], dim=1))  # connexion directe
        return self.tete(d1)

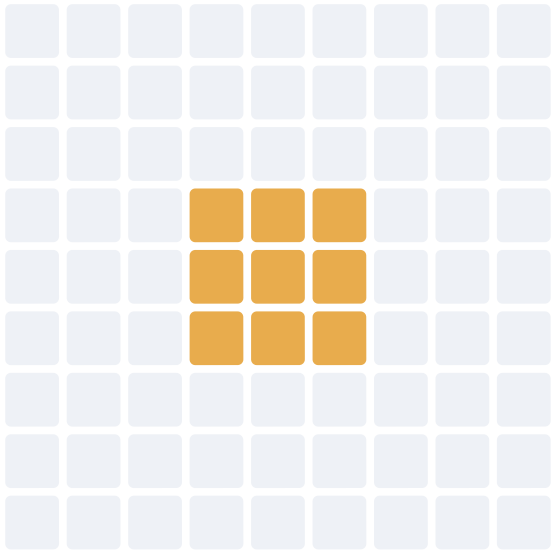
print(MiniUNet(n_classes=2)(torch.randn(1, 3, 64, 64)).shape)
# torch.Size([1, 2, 64, 64]) : un score par classe, à la taille d'entrée
```

- **Encodeur** : bloc de convolutions, puis **MaxPool2d** ; **décodeur** : **ConvTranspose2d** de pas 2
- La connexion directe tient en une ligne : **torch.cat([d1, e1], dim=1)**, sur les canaux
- Après la concaténation : 16 + 16 = **32** canaux, d'où bloc(32, 16)
- Sortie **(1, 2, 64, 64)** : un score par classe et par pixel, à la taille de l'entrée

Source : Version réduite de Ronneberger et al., 2015 (deux niveaux au lieu de cinq).

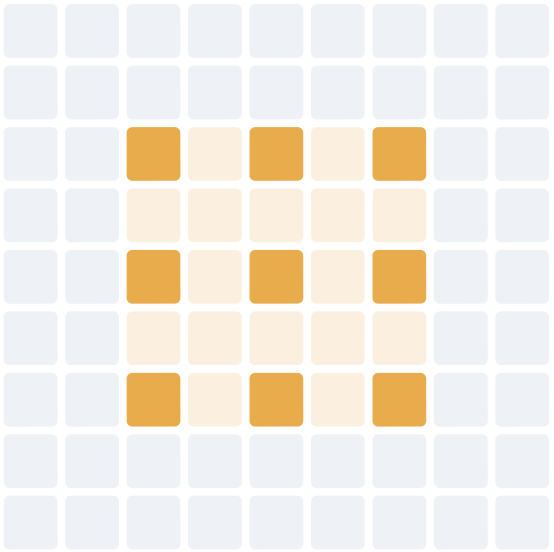
La convolution dilatée

Pourquoi : la brique de DeepLab : voir plus large sans perdre de résolution.



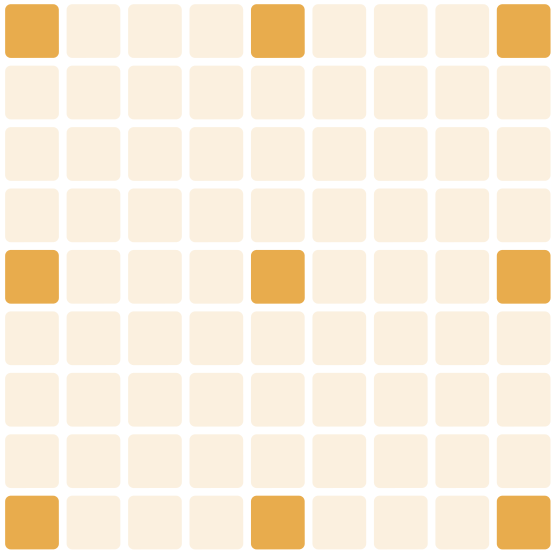
taux 1

9 poids, champ 3 × 3



taux 2

9 poids, champ 5 × 5



taux 4

9 poids, champ 9 × 9

- **Même nombre de poids** (9)
- **Même taille de sortie** (marge égale au taux)
- Un **champ récepteur** plus large, sans pooling

Segmenter demande du **contexte** et une **résolution fine** : le pooling donne le premier en détruisant la seconde ; la convolution dilatée donne le contexte sans réduire la résolution.

Source : Chen et al., Rethinking Atrous Convolution (DeepLabv3), 2017 ; exemple calculé avec PyTorch.

Convolution dilatée en PyTorch

Pourquoi : un seul argument, dilation, et le champ s'élargit sans changer la taille de sortie.

```
import torch
import torch.nn as nn

x = torch.zeros(1, 1, 15, 15)
x[0, 0, 7, 7] = 1                          # un seul pixel allumé
for d in (1, 2, 4):
    conv = nn.Conv2d(1, 1, 3, dilation=d, padding=d, bias=False)
    nn.init.ones_(conv.weight)
    y = conv(x)
    lignes = (y[0, 0].sum(dim=1) > 0).nonzero().flatten().tolist()
    print(d, tuple(y.shape[2:]), conv.weight.numel(), lignes)
# 1 (15, 15) 9 [6, 7, 8]
# 2 (15, 15) 9 [5, 7, 9]      même taille, 9 poids,
# 4 (15, 15) 9 [3, 7, 11]     champ plus large
```

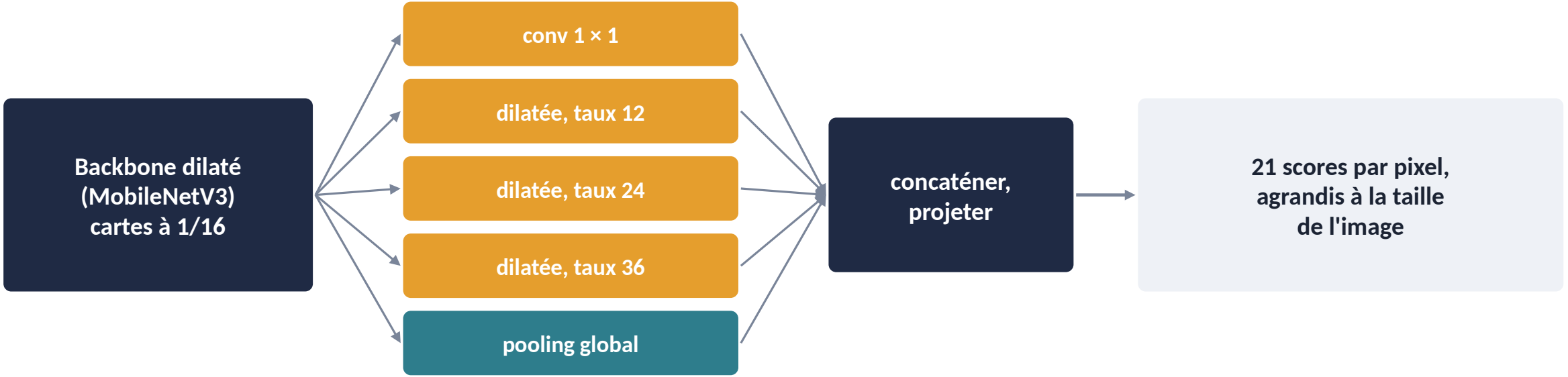
- **dilation=d** espace les 9 poids de d pixels ; **padding=d** garde la taille de sortie
- Un seul pixel allumé (ligne 7) : la sortie touche les lignes [6, 7, 8], puis [5, 7, 9], puis [3, 7, 11]
- Toujours **9 poids**, une sortie **15 × 15** : seul le champ s'élargit

Source : Documentation PyTorch : torch.nn.Conv2d (argument dilation).

DeepLab et son module ASPP

Pourquoi : le modèle sémantique de l'étape 8.

DeepLab est une famille de modèles de segmentation sémantique (Chen et al., Google). Son module **ASPP** (**Atrous Spatial Pyramid Pooling**) applique **en parallèle** plusieurs convolutions dilatées (« atrous » : à trous) d'écarts différents : chaque pixel est regardé **à plusieurs distances** à la fois.



Chaque branche de l'**ASPP** voit la même carte à une **échelle** différente : de petits et de grands objets dans la même passe.

Source : Chen et al., Rethinking Atrous Convolution for Semantic Image Segmentation, 2017.

DeepLab en PyTorch

Pourquoi : le modèle de l'étape 8, et la forme de sa sortie.

```python
import torch
from torchvision.models.segmentation import deeplabv3_mobilenet_v3_large

modele = deeplabv3_mobilenet_v3_large(weights="DEFAULT").eval()
x = torch.randn(1, 3, 416, 416)      # image normalisée comme ImageNet
with torch.no_grad():
    scores = modele(x)["out"]
print(scores.shape)         # torch.Size([1, 21, 416, 416]) : 20 + fond
masque = scores.argmax(dim=1) == 15  # classe 15 : personne
print(masque.shape)                  # torch.Size([1, 416, 416])

aspp = modele.classifier[0]          # le module ASPP
print([m[0].dilation for m in aspp.convs[1:-1]])   # ses taux
# [(12, 12), (24, 24), (36, 36)]
```

- Le modèle de l'étape 8 : **DeepLabV3 MobileNetV3**, pré-entraîné sur COCO avec les classes de Pascal VOC
- Sortie **out** : **(N, 21, H, W)**, à la taille de l'entrée
- **argmax** sur les classes, puis **== 15** : le masque des personnes
- **classifier[0]** est l'ASPP : taux de dilatation **12, 24, 36**

Source : Documentation torchvision : deeplabv3_mobilenet_v3_large (mIoU 60,3 sur COCO val2017, étiquettes VOC).

La segmentation sémantique : à retenir

1. Segmenter, c'est **classer chaque pixel** : la sortie a la forme (N, C, H, W), on prend l'argmax sur C ; la mesure est le **mIoU**, moyenne des IoU par classe.

2. **U-Net** : un encodeur réduit l'image, un décodeur la remonte, et des **connexions directes** réinjectent les détails fins.

3. **DeepLab** : des **convolutions dilatées** élargissent le champ de vision sans réduire la résolution, et l'**ASPP** combine plusieurs taux de dilatation.

Dans le TD : Étape 8 : DeepLab sur les images de chantier, classe 15 (personne).

Pour approfondir : d2l, section 14.9 : Semantic Segmentation and the Dataset ; Ronneberger et al., 2015 : l'article U-Net (arXiv)

SAM : définition

Définition

SAM (Segment Anything Model) segmente un objet à partir d'une indication, un point ou une boîte, sans entraînement propre à la tâche. Il rend un masque, sans classe.

Le problème que cela résout

Obtenir des masques précis sans annoter des milliers d'images au pixel : combiné aux boîtes d'un détecteur, il donne un masque par objet.

Ce qui entre, ce qui sort

Entrée : une image et une indication (points, boîtes). Sortie : un masque binaire (H, W) par indication.

Ce que l'on va voir, dans l'ordre

2. Un masque par objet : Mask R-CNN, ou YOLO puis SAM
3. Comment SAM segmente à partir d'une indication
4. SAM en PyTorch
5. À retenir

Dans le TD : Étape 8 : les boîtes « Person » du modèle fine-tuné, données à MobileSAM.

Pour en savoir plus : Kirillov et al., 2023 : l'article Segment Anything (arXiv) ; Documentation Ultralytics : MobileSAM

Un masque par objet : Mask R-CNN, ou YOLO puis SAM

Pourquoi : compter et séparer les objets, ce que la segmentation sémantique ne fait pas.

La **segmentation d'instances** donne un masque **séparé pour chaque objet** : chaque personne a le sien. Deux façons de l'obtenir : **un seul modèle** qui sort boîtes et masques (Mask R-CNN), ou **deux modèles en chaîne** : YOLO trouve les boîtes, puis SAM dessine le masque dans chaque boîte.

Mask R-CNN

- Faster R-CNN + une branche masque dans la tête
- un masque 28 × 28 par région, agrandi
- s'entraîne d'un bloc : il faut des masques annotés

YOLO puis SAM

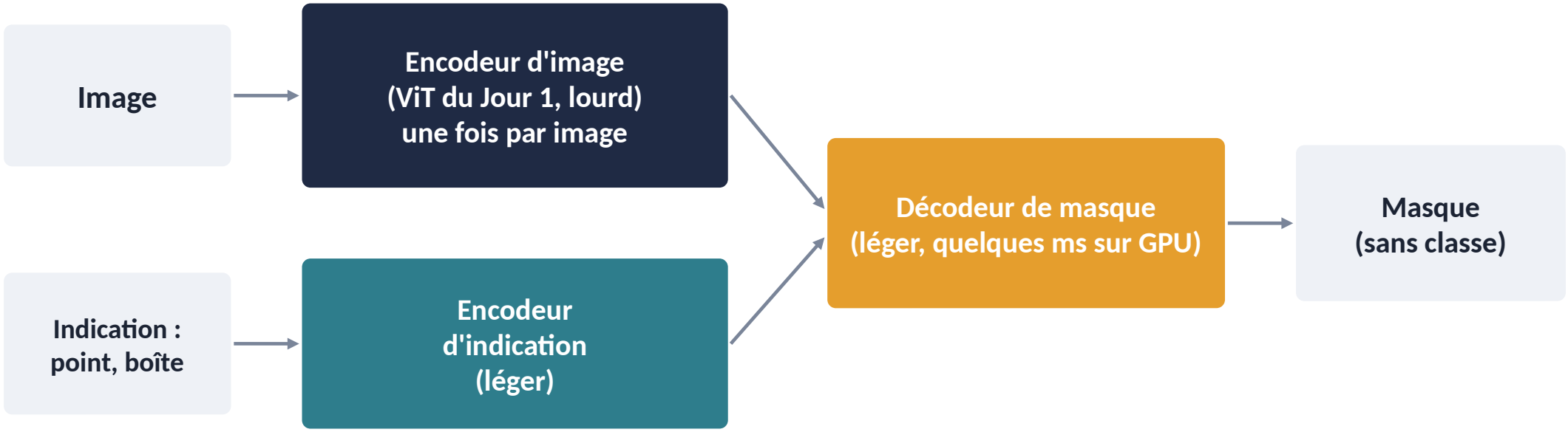
- YOLO donne les boîtes et les classes
- SAM (Segment Anything Model) transforme chaque boîte en masque, sans entraînement
- pas de masques à annoter ; deux modèles à exécuter

YOLO puis SAM : c'est le pipeline de l'**étape 8**, et un outil courant pour **pré-annoter** des données (auto_annotate dans Ultralytics).

Source : He et al., Mask R-CNN, ICCV 2017 ; Ultralytics, auto_annotate (détecteur puis SAM).

Comment SAM segmente à partir d'une indication

Pourquoi : comprendre l'outil de l'étape 8, présenté en détail au Jour 4.



- Entraîné sur **SA-1B**, le jeu de données construit pour SAM : 1,1 milliard de masques, 11 millions d'images
- Plusieurs indications sur la même image : l'encodeur lourd ne tourne **qu'une fois**
- **MobileSAM** : encodeur TinyViT distillé (un petit réseau entraîné à imiter le gros) d'environ **6 M** de paramètres au lieu de plus de 600 M (environ 10 M au total) ; même décodeur
- Il ne connaît **pas les classes** : où est l'objet, pas ce qu'il est

Source : Kirillov et al., Segment Anything, ICCV 2023 ; Zhang et al., MobileSAM, 2023.

SAM en PyTorch

Pourquoi : le pipeline YOLO puis SAM de l'étape 8, en quelques lignes.

```python
from ultralytics import SAM, YOLO

yolo = YOLO("yolov8n.pt")
r = yolo.predict("astronaute.jpg", conf=0.5, classes=[0], verbose=False)
boites = r[0].boxes.xyxy                    # les personnes détectées

sam = SAM("mobile_sam.pt")                  # MobileSAM, la version légère
m = sam("astronaute.jpg", bboxes=boites.tolist(), verbose=False)
masques = m[0].masks.data
print(boites.round(), masques.shape)        # une boîte -> un masque (H, W)
# tensor([[  0.,  18., 367., 511.]]) torch.Size([1, 512, 512])
print(masques[0].float().mean().round(decimals=3))   # tensor(0.4070)
```

- YOLO donne les boîtes **personne** (classes=[0]) ; SAM les reçoit par **bboxes=**
- Sortie : **un masque (H, W)** par boîte, à la taille de l'image
- Le masque couvre **41 %** de l'image, la boîte **69 %** (prédite ici à 640 px) : elle englobe aussi du fond
- Au TD (étape 8) : boîtes **Person** (classes=[6]) du modèle fine-tuné, conf=0.3, puis MobileSAM

Source : Documentation Ultralytics : SAM et MobileSAM.

SAM : à retenir

1. La **segmentation d'instances** donne un masque par objet : Mask R-CNN l'apprend, YOLO + SAM la compose à partir de boîtes.

2. **SAM** : un encodeur d'image lourd, calculé une fois par image, puis un décodeur léger qui rend un masque pour chaque indication ; **MobileSAM** allège l'encodeur.

3. SAM ne connaît **pas les classes** : c'est le détecteur qui dit ce qu'est chaque objet. Son fonctionnement sera détaillé au Jour 4.

Dans le TD : Étape 8 : les boîtes « Person » du modèle fine-tuné, données à MobileSAM.

Pour approfondir : Kirillov et al., 2023 : l'article Segment Anything (arXiv) ; Documentation Ultralytics : MobileSAM

Retour au cas d'étude

Pourquoi : répondre aux trois questions du matin.

1

Comment localiser les objets ?

étapes 2 et 4

des boîtes avec un score : YOLO prédit en une passe sur trois grilles, puis seuil de confiance et NMS par classe

2

Comment savoir si le détecteur est bon ?

étapes 5 à 7

précision, rappel, AP, mAP sur la validation ; fine-tuning de YOLOv8n ; baseline couleur ; seuil choisi selon le coût d'un oubli, mesuré une fois sur le test

3

Faut-il des boîtes ou des masques ?

étape 8

pour compter et alerter, des boîtes suffisent ; pour la forme exacte, DeepLab (sémantique) ou YOLO puis SAM (par objet)

TD · étape 10

Synthèse

Pourquoi : les réponses à garder en tête pour le TD.

1. **Une détection** : une boîte, une classe, un score. IoU pour le recouvrement, NMS pour les doublons, toutes deux classe par classe chez YOLO.

2. **YOLO** : un CNN, puis une tête qui prédit en chaque point de trois grilles les distances aux bords et un score par classe ; seuil puis NMS.

3. **Mesurer** : apparier par score décroissant ; précision et rappel à un seuil ; AP et mAP sur tous les seuils ; le seuil final se choisit selon le coût métier.

4. **Fine-tuner** : la tête passe à nos classes, tout le réseau se réentraîne ; environ 2 min 30 par époque sur CPU, et le coût de l'annotation.

5. **Segmenter** : une classe par pixel (U-Net, DeepLab, mesurés par le mIoU), ou un masque par objet (Mask R-CNN, YOLO puis SAM).

Ouverture : la suite du module

Pourquoi : ce que vous avez appris aujourd'hui sert dans les journées suivantes.

Jour 3	**Séries temporelles** repérer des anomalies dans les capteurs d'une pompe : même logique de seuil d'alerte, selon le coût des erreurs
Jour 4	**Modèles de fondation** SAM revient comme foundation model, avec CLIP et DINOv2
Jour 6	**Mise en production** exporter, quantifier et mesurer un détecteur sur la machine cible
Projet	**Brique 2** localiser les défauts de NEU-DET (photos de tôles d'acier) avec YOLO, comme à l'étape 11

Déroulé du TD 2

Pourquoi : chaque étape du TD applique une partie du cours.

Étape 2

Coder l'IoU et la NMS (TODO 1 et 2)

partie 1

Étape 3

Le jeu Construction-PPE

parties 1 et 4

Étape 4

YOLOv8n de COCO face au chantier ; votre NMS

parties 1 et 2

Étape 5

Baseline couleur et évaluation (TODO 3 et 4)

partie 3

Étape 6

Fine-tuning court ; analyse du modèle fourni

parties 3 et 4

Étape 7

Choisir le seuil (TODO 5) ; une mesure sur le test

partie 3

Étape 8

Segmentation : DeepLab, YOLO puis SAM

partie 6

Étape 9

Mini-application

partie 2

Étape 10

Fiche de décision

synthèse

Étape 11

Brique 2 : localiser les défauts d'acier

parties 3 et 4



Étapes 0 et 1 : vérifier l'installation et revoir le cours en bref. Pendant le TD : la pastille orange « TD · étape » en haut à gauche des diapositives indique où elles servent.

Pour aller plus loin

Pourquoi : des cours et documentations gratuits, qui reprennent les notions du jour.

Dive into Deep Learning (d2l), chapitre 14	boîtes, ancres, IoU, NMS, SSD (un autre détecteur one-stage), R-CNN, segmentation sémantique, avec code	anglais
Documentation Ultralytics	YOLOv8, entraînement, métriques, export, SAM et MobileSAM	anglais
Tutoriel torchvision : Object Detection Finetuning	fine-tuner Faster R-CNN et Mask R-CNN pas à pas	anglais
Ultralytics : YOLO Performance Metrics	précision, rappel, AP, mAP, illustrés	anglais
CS231n (Stanford), site du cours	séance Detection and Segmentation : panorama des deux tâches	anglais
Segment Anything (démo et article)	essayer SAM dans le navigateur	anglais

Références

- Construction-PPE, Ultralytics (jeu de données du TD 2).
- Everingham et al., The PASCAL Visual Object Classes (VOC) Challenge, IJCV 2010.
- Lin et al., Microsoft COCO: Common Objects in Context, ECCV 2014.
- Redmon et al., You Only Look Once: Unified, Real-Time Object Detection, CVPR 2016.
- Ren et al., Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks, NeurIPS 2015.
- Lin et al., Feature Pyramid Networks for Object Detection, CVPR 2017.
- Ge et al., YOLOX: Exceeding YOLO Series in 2021, 2021.
- Li et al., Generalized Focal Loss, NeurIPS 2020.
- Zheng et al., Distance-IoU Loss, AAAI 2020.
- Long, Shelhamer et Darrell, Fully Convolutional Networks for Semantic Segmentation, CVPR 2015.
- Ronneberger, Fischer et Brox, U-Net: Convolutional Networks for Biomedical Image Segmentation, MICCAI 2015.
- Chen et al., Rethinking Atrous Convolution for Semantic Image Segmentation (DeepLabv3), 2017.
- He et al., Mask R-CNN, ICCV 2017.
- Kirillov et al., Panoptic Segmentation, CVPR 2019.
- Kirillov et al., Segment Anything, ICCV 2023.
- Zhang et al., Faster Segment Anything: Towards Lightweight SAM for Mobile Applications (MobileSAM), 2023.

Chiffres de modèles : pages Ultralytics (YOLOv8) et métadonnées des poids torchvision. Code exécuté avec les versions de uv.lock (PyTorch 2.14.1, Ultralytics 8.4.175).